

LORE

Algorithm Specifications And Supporting Documentation

Zhongxiang Zheng, Anyu Wang, Chunhuan Zhao, Guangwu Xu,
Sibo Feng, Jingtao Wang, Wanshan Xu, Yuhang Sun, Yingxue Ma,
Huayun Tang, Shuang Sun, Shuaigang Li

Table of Contents

1	Introduction	1
1.1	Design Rationale	1
1.2	Feature Statements	3
2	Preliminaries	4
2.1	Notations	4
2.2	Lattice	4
2.3	LWE Problem	5
2.4	LWR Problem	6
2.5	Chinese Remainder Theorem	6
2.6	The CRT_LWR Problem	7
2.7	Reduction from CRT_LWR to LWR	8
2.8	CRT_MLWR Problem	8
2.9	Public Key Encryption and Key Encapsulation Mechanism	9
2.10	Encoding	10
3	PKE and KEM Schemes	11
3.1	Overview	11
3.2	Core Strategy: Variable Modulus and CRT Compression	14
3.3	Public Key Encryption Algorithm	15
3.4	Key Encapsulation Mechanism Algorithm	17
3.5	Specification	19
4	Security Analysis	20
4.1	Completeness and DFR	20
4.2	Security Proof	22
4.3	Cost of known attacks	24
5	Performance	25
5.1	Parameter Sets	25
5.2	Performance Analysis	26

1 Introduction

Lore is a post-quantum key encapsulation mechanism (KEM) that achieves IND-CCA2 security based on the hardness of the Module-LWR problem. Departing from conventional designs that use a single modulus across all security levels, Lore introduces two synergistic techniques: **Variable Modulus** and **CRT Compression**. The Variable Modulus method employs a composite modulus tq (with fixed $q = 257$ and a security-dependent small t) to decouple the decryption failure rate (DFR) from the modulus size, allowing the DFR to be set strictly below $2^{-\lambda}$ for each target security level λ without wasteful over-parameterization. CRT Compression leverages the Chinese Remainder Theorem to represent elements of $\mathbb{Z}_{tq}$ by their residues in $\mathbb{Z}_q$ and $\mathbb{Z}_t$, and applies controlled rounding steps that significantly shrink public-key and ciphertext sizes while introducing only a small, predictable error whose distribution is analogous to that of LWR. The security of the resulting CRT-LWR problem is formally reduced to that of standard LWR. We provide a full security proof in the (quantum) random oracle model and present concrete parameter sets for 128-, 256-, 384-, and 512-bit classical security that meet the requirements of NGCC [32]. A reference implementation demonstrates that Lore achieves a combined public-key and ciphertext size 25%–30% smaller and a total cost 20%–24% smaller than Kyber at equivalent security levels, while maintaining a DFR tightly aligned with the target security bound (e.g., 2^{-266} for 256-bit security). In summary, Lore offers several advantages:

- **Compactness:** By leveraging CRT Compression, Lore achieves a significantly smaller combined public key and ciphertext size (bandwidth) compared to prior LWE schemes, as demonstrated in Table 1.
- **Overall performance:** Lore achieves a better total cost (as defined in the NIST report [1], where transmission overhead is taken into account) compared to Kyber, as shown in Table 4.
- **Low DFR matching security level:** Lore’s DFR is unconditionally below $2^{-\lambda}$, satisfying the security reduction of the FO transform, eliminating the need for query limitations and strengthening resistance to decryption-failure attacks.

1.1 Design Rationale

The Lore PKE/KEM scheme is designed with the primary goals of achieving compact bandwidth and a Decryption Failure Rate (DFR) that tightly matches the target security level.

To date, many lattice-based public-key encryption (PKE) and key encapsulation mechanism (KEM) schemes have been built upon the Learning with Errors (LWE) problem following the LPR framework [30]. LPR-based encryption

Table 1. Comparison of bandwidth ($|\text{PK}| + |\text{CT}|$ in bytes) between Lore and other lattice-based KEMs.

Scheme	Level	Assumption	DFR	$ \text{PK} $	$ \text{CT} $	Bandwidth
BAT-512 [20]		NTRU	2^{-146}	521	473	994
DAWN-alpha-512 [26]		NTRU	2^{-133}	615	436	1051
Lore-128 (This work)		LWE	2^{-128}	545	641	1186
NEV-512 [40]		NTRU	2^{-138}	615	615	1230
LAC-128-v3a [28]	1	LWE	2^{-151}	544	704	1248
Aigis-128 [41]		LWE	2^{-82}	672	672	1344
SMAUG-T-128 [14]		LWE	2^{-118}	672	672	1344
LightSaber [18]		LWE	2^{-120}	672	736	1408
ML-KEM-2 (Kyber2) [37]		LWE	2^{-138}	800	768	1568
DAWN-alpha-1024 [26]		NTRU	2^{-163}	1229	973	2202
Lore-256 (This work)		LWE	2^{-266}	1058	1153	2211
BAT-1024 [20]		NTRU	2^{-166}	1230	1006	2236
NEV-1024 [40]		NTRU	2^{-152}	1229	1229	2458
LAC-256-v3a [28]	5	LWE	2^{-302}	1056	1464	2520
FireSaber [18]		LWE	2^{-165}	1312	1472	2784
SMAUG-T-256 [14]		LWE	2^{-194}	1440	1376	2816
Aigis-256 [41]		LWE	2^{-211}	1472	1536	3008
ML-KEM-5 (Kyber5) [37]		LWE	2^{-174}	1568	1568	3136

offers good efficiency and compact bandwidth together with a complete security proof. CRYSTALS-KYBER [6] adopts Module-LWE instead of Ring-LWE to achieve better scalability and high efficiency. Both CRYSTALS-KYBER and SMAUG-T [14]—selected for standardization by NIST and KpqC, respectively—are based on the module version of the LPR structure.

Although extensively studied, we believe that current schemes built on the LPR structure still exhibit two aspects that need improvement. First, in terms of total cost, when transmission overhead is accounted for (e.g., 2000 cycles/byte for general-purpose use, following the NIST report [1]), the contribution of algorithmic efficiency to the total cost is extremely small (less than 10%) compared to the transmission component. Therefore, reducing bandwidth at the expense of a modest efficiency loss can substantially enhance the overall performance of the schemes. Second, in KEM designs the DFR directly impacts the security guarantees of the Fujisaki–Okamoto (FO) transform. As shown by Hofheinz, Hövelmanns, and Kiltz [25], the concrete security bound of an FO-derived KEM includes an additive term equal to the decryption error probability of the underlying PKE. Hence, to achieve λ -bit security, the DFR must be negligible (typically $< 2^{-\lambda}$). Cini et al. [15] emphasize that decryption errors are critical because the FO proof relies on the error probability being bounded. A non-negligible DFR invalidates standard FO proofs and may expose the scheme to failure-exploitation attacks. The NIST call for proposals [33] notes that while security estimates may assume at most 2^{64} decryption queries per key pair, attacks involving more queries cannot be ruled out. Nevertheless, most existing schemes [6, 14] analyze DFR under this limited-query assumption, neglecting the possibility of larger-scale failure exploitation. Moreover, the exact security impact of a high DFR remains underexplored, and recent attacks exploiting decryption failures [11, 16, 22, 23, 39] have raised serious concerns about the validity of security claims when the DFR exceeds the target level. A scheme whose DFR matches its security strength therefore eliminates the need for query limits and strengthens its resistance to failure-based attacks.

To address these issues, we design Lore, which simultaneously achieves compact bandwidth, low total cost, and a Decryption Failure Rate (DFR) that tightly matches the target security level.

1.2 Feature Statements

- **Security foundation:** Lore relies on the hardness of the MLWE/MLWR problem, a well-studied lattice assumption.
- **DFR matching security:** The DFR for each parameter set is strictly below $2^{-\lambda}$, satisfying the security reduction of the FO transform, eliminating the need for query limitations and strengthening resistance to decryption-failure attacks.

- **Compact bandwidth:** The sum of public key and ciphertext sizes is 25%–30% smaller than Kyber at equivalent security levels (see Table 1), confirming that the fixed-modulus approach inherently limits both DFR and bandwidth when scaling security. This makes Lore one of the most bandwidth-efficient lattice-based designs to date.
- **Implementation friendliness:** The use of fixed $q = 257$ and small t allows efficient NTT and CRT-based arithmetic, simplifying implementation.
- **Computational overhead:** Due to separate operations in $\mathcal{R}_q$ and $\mathcal{R}_t$, Lore is computationally slower than Kyber in pure computation. However, when the estimated transmission cost of 2000 cycles per byte (as suggested by the NIST status report [1]) is factored into the total cost, Lore’s substantial bandwidth advantage more than compensates for its computational overhead. At the 128-bit security level, Lore-128 reduces the total cost by approximately 20% compared to Kyber-512. At the 256-bit security level, Lore-256 outperforms Kyber-1024 by roughly 24%.

2 Preliminaries

2.1 Notations

Matrices are denoted in bold uppercase letters (e.g., $\mathbf{A}$), and vectors in bold lowercase (e.g., $\mathbf{y}$ or $\mathbf{z}_1$). The i -th component of a vector is written as y_i . Let $\mathcal{R} = \mathbb{Z}[x]/(x^n + 1)$ be a polynomial ring, and for any positive integer q define $\mathcal{R}_q = \mathbb{Z}_q[x]/(x^n + 1)$. For $\mathbf{y} = (y_0 + y_1x + \dots + y_{n-1}x^{n-1}, \dots, y_{(k-1)n} + \dots + y_{kn-1}x^{n-1})^\top \in \mathcal{R}^k$, we define its ℓ_∞ -norm as $\|\mathbf{y}\|_\infty = \max_{0 \leq i < kn} |y_i|$, where the coefficients are taken as integers in the range $[-\lfloor q/2 \rfloor, \lfloor q/2 \rfloor]$.

2.2 Lattice

An m -dimensional *lattice* is a discrete additive subgroup in $\mathbb{R}^m$ which can be represented as the set of integer linear combination of n linearly independent vectors $\{\mathbf{b}_1, \dots, \mathbf{b}_n\}$, i.e.

$$\mathcal{L}(\mathbf{B}) = \left\{ \sum_{i=1}^n x_i \mathbf{b}_i \mid x_i \in \mathbb{Z}, \forall i \in [1, \dots, n] \right\},$$

where $\mathbf{B} = [\mathbf{b}_1, \dots, \mathbf{b}_n]$ is called a *basis* of $\mathcal{L}$ which is not unique, n ($n \leq m$) is the *rank* of the lattice, a lattice is called full-rank if $m = n$. The determinant of $\mathcal{L}$ is defined as

$$\det(\mathcal{L}) = \sqrt{\det(\mathbf{B}^\top \mathbf{B})}.$$

The quantity $\det(\mathcal{L})$ is invariant regardless of the choice of $\mathbf{B}$. The *dual lattice* $\mathcal{L}^*$ is defined as

$$\mathcal{L}^* = \{\mathbf{w} \in \mathbb{R}^m \mid \forall \mathbf{v} \in \mathcal{L}, \langle \mathbf{w}, \mathbf{v} \rangle \in \mathbb{Z}\}.$$

q -ary lattice As a kind of important lattices in lattice-based cryptography, a q -ary lattice refers to the lattice such that $q\mathbb{Z}^n \subseteq \mathcal{L} \subseteq \mathbb{Z}^n$ where q is an integer.

Two types of q -ary lattices frequently used in lattice cryptography are defined as follows with respect to an $n \times m$ matrix $\mathbf{B} \in \mathbb{Z}_q^{n \times m}$,

$$\begin{aligned}\mathcal{L}_q(\mathbf{B}) &= \{\mathbf{y} \in \mathbb{Z}^m \mid \mathbf{y} = \mathbf{B}^\top \mathbf{x} \bmod q, \mathbf{x} \in \mathbb{Z}^n\}, \\ \mathcal{L}_q^\perp(\mathbf{B}) &= \{\mathbf{y} \in \mathbb{Z}^m \mid \mathbf{B}\mathbf{y} = \mathbf{0} \bmod q\}.\end{aligned}$$

2.3 LWE Problem

Learning with Errors (LWE) problem was proposed by Regev [34] in 2005 and has been widely used in the construction of lattice-based cryptography. We first introduce some definitions in order to describe LWE problems.

Definition 2.1 (LWE distribution). Let $n \geq 1$, $q \geq 2$ and χ be an error distribution over $\mathbb{Z}_q$, given a secret vector $\mathbf{s} \in \mathbb{Z}_q^n$, the LWE distribution $L_{\mathbf{s}, \chi}$ over $\mathbb{Z}_q^n \times \mathbb{Z}_q$ is sampled by choosing $\mathbf{a} \leftarrow U(\mathbb{Z}_q^n)$ and $e \leftarrow \chi$, and outputting $(\mathbf{a}, b = \langle \mathbf{a}, \mathbf{s} \rangle + e \bmod q)$.

The LWE problem has a search version and a decision version, which are defined as follows.

Definition 2.2 (Search-LWE). Given m independent samples $(\mathbf{a}_i, b_i) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ drawn from $L_{\mathbf{s}, \chi}$ for a fixed $\mathbf{s}$, the goal is to recover $\mathbf{s}$.

In the following part of this paper, we denote $\mathbf{A} \in \mathbb{Z}_q^{m \times n}$ to be the matrix formed by m rows $\{\mathbf{a}_i\}_{i=1}^m$ and $\mathbf{b} = (b_1, b_2, \dots, b_m)^\top \in \mathbb{Z}_q^m$, where $b_i = \langle \mathbf{a}_i, \mathbf{s} \rangle + e_i \bmod q$.

Definition 2.3 (Decision-LWE). Given m independent samples $(\mathbf{A}, \mathbf{b}) \in \mathbb{Z}_q^{m \times n} \times \mathbb{Z}_q^m$, the goal is to distinguish whether they are drawn from $L_{\mathbf{s}, \chi}$ (for some fixed but unknown $\mathbf{s} \leftarrow \chi^n$) or from the uniform distribution over $\mathbb{Z}_q^{m \times n} \times \mathbb{Z}_q^m$.

We denote the LWE problem as $\text{LWE}_{m,n,\chi,q}$ where the secret vector $\mathbf{s} \leftarrow \chi^n$ and the error vector $\mathbf{e} \leftarrow \chi^m$ are both drawn independently from the same distribution χ . Applebaum et al. [5] proved that this variant is as hard as the standard LWE problem (where $\mathbf{s}$ is uniform). Besides, to make LWE more practical in cryptography, variants of LWE problems (e.g., Ring-LWE and Module-LWE) have been investigated. More details of these variants can be found in [29]. We remark that any $\text{MLWE}_{n,q,k,l,\chi}$ instance can be viewed as an $\text{LWE}_{nk,nl,\chi,q}$ instance, where n denotes the dimension of the polynomial ring.

Definition 2.4 (Standard LWE Encryption). A public-key encryption scheme $\text{PKE} = (\text{KeyGen}, \text{Encryption}, \text{Decryption})$ based on LWE is defined as follows:

- **KeyGen:** Choose $\mathbf{A} \leftarrow U(\mathbb{Z}_q^{m \times n})$, sample $\mathbf{s} \leftarrow \chi^n$, $\mathbf{e} \leftarrow \chi^m$, and set $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \bmod q$. Output $\text{pk} = (\mathbf{A}, \mathbf{b})$, $\text{sk} = \mathbf{s}$.

- Encryption: Given a message bit $\mu \in \{0, 1\}$, sample $\mathbf{s}' \leftarrow \chi^m$, $\mathbf{e}' \leftarrow \chi^n$, $e'' \leftarrow \chi$. Compute $\mathbf{C}_u = \mathbf{A}^\top \mathbf{s}' + \mathbf{e}' \bmod q$, $C_v = \mathbf{b}^\top \mathbf{s}' + e'' + \lfloor q/2 \rfloor \mu \bmod q$. Output $\mathbf{c} = (\mathbf{C}_u, C_v)$.
- Decryption: Compute $C'_v = (C_v - \mathbf{C}_u^\top \mathbf{s}) \bmod q$. If $|C'_v - q/2| < q/4$, output 1; otherwise output 0.

Correctness holds if the error term $E = \mathbf{e}^\top \mathbf{s}' - \mathbf{e}'^\top \mathbf{s} + e''$ satisfies $|E| < q/4$.

2.4 LWR Problem

The Learning with Rounding (LWR) scheme, which was first introduced by Banerjee et al. [7] and independently formalized by Alwen et al. [4], is a variant of the Learning with Errors problem. Its security is provably as hard as the LWE problem under polynomial modulus reductions, while eliminating the need for Gaussian sampling—a feature that significantly enhances computational efficiency and simplifies practical implementations.

More formally, the LWR-problem is defined via the following rounding function for integers $N \geq p \geq 2$:

$$\lfloor \cdot \rfloor_p : \mathbb{Z}_N \rightarrow \mathbb{Z}_p : x \mapsto \lfloor (p/N) \cdot x \rfloor,$$

Definition 2.5 (Decision-LWR $_{m,n,p,N,\chi}$). Let $n \geq 1$, $N \geq p \geq 2$, given a secret vector $\mathbf{s} \in \mathbb{Z}_N^n$ sampled from χ , uniformly sample a public matrix $\mathbf{A} \in \mathbb{Z}_N^{m \times n}$, computing the rounded product $\mathbf{b} = \lfloor \mathbf{A} \cdot \mathbf{s} \rfloor_p$, the tuple $(\mathbf{A}, \mathbf{b})$ constitutes the LWR sample. Similar to **Decision-LWE**, we now additionally sample a uniformly random vector $\mathbf{u} \leftarrow U(\mathbb{Z}_N^n)$, the goal of **Decision-LWR** states that $(\mathbf{A}, \mathbf{b})$ is computationally indistinguishable from $(\mathbf{A}, \lfloor \mathbf{u} \rfloor_p)$.

Definition 2.6 (Decision-MLWR $_{n,p,N,k,l,\chi}$). For positive integers n, p, N, k, l, χ with $N \geq p \geq 2$ and the dimension n of $\mathcal{R}$, we say that the advantage of an adversary $\mathcal{A}$ solving the **Decision-MLWR $_{n,p,N,k,l,\chi}$** problem is

$$\begin{aligned} \text{Adv}_{n,p,N,k,l,\chi}^{\text{MLWR}}(\mathcal{A}) = & \left| \Pr[\mathcal{A}(\mathbf{A}, \lfloor \mathbf{b} \rfloor_p) = 1 \mid \mathbf{A} \leftarrow \mathcal{R}_N^{k \times l}, \mathbf{b} \leftarrow \mathcal{R}_N^k] \right. \\ & \left. - \Pr[\mathcal{A}(\mathbf{A}, \lfloor \mathbf{A} \cdot \mathbf{s} \rfloor_p) = 1 \mid \mathbf{A} \leftarrow \mathcal{R}_N^{k \times l}, \mathbf{s} \leftarrow \chi^l] \right|. \end{aligned}$$

2.5 Chinese Remainder Theorem

Let $m_1, \dots, m_r$ be pairwise coprime positive integers, and let $M = \prod_{i=1}^r m_i$. For any integers $a_1, \dots, a_r$, the system of congruences

$$x \equiv a_i \pmod{m_i} \quad (1 \leq i \leq r)$$

has a unique solution modulo M . This solution is given by

$$x \equiv \sum_{i=1}^r a_i \frac{M}{m_i} \left(\left(\frac{M}{m_i} \right)^{-1} \bmod m_i \right) \bmod M.$$

Consequently, the map

$$\mathbb{Z}_M \longrightarrow \mathbb{Z}_{m_1} \times \cdots \times \mathbb{Z}_{m_r}, \quad x \bmod M \mapsto (x \bmod m_1, \dots, x \bmod m_r)$$

is a ring isomorphism.

In this work we apply the CRT to the modulus tq , where t and q are coprime. For any $x \in \mathbb{Z}_{tq}$, we write $x_t = x \bmod t$ and $x_q = x \bmod q$. Then x is uniquely determined by the pair (x_t, x_q) via

$$x \equiv x_t \cdot q \cdot (q^{-1} \bmod t) + x_q \cdot t \cdot (t^{-1} \bmod q) \bmod tq.$$

2.6 The CRT_LWR Problem

For any integer $t \geq 2$, define the symmetric residue system

$$\text{Sym}(t) = \left\{ -\left\lfloor \frac{t-1}{2} \right\rfloor, \dots, \left\lceil \frac{t-1}{2} \right\rceil \right\},$$

which is a complete set of representatives for $\mathbb{Z}_t$. For $x \in \mathbb{Z}_t$, denote by $x \bmod^{\pm t}$ the unique element of $\text{Sym}(t)$ congruent to x modulo t .

Given a non-empty set $S_R \subset \mathbb{Z}_t$, define the rounding function

$$\lfloor x \rfloor_{S_R} = \arg \min_{x' \in S_R} \text{dist}(x', x), \quad \text{dist}(x', x) = \min\{|x' - x|, t - |x' - x|\},$$

where distances are computed by lifting to representatives in $\{0, 1, \dots, t-1\}$. Ties are broken by choosing the $x' \in S_R$ that minimizes $(x - x') \bmod t$ (where the modulo operation returns a value in $\{0, 1, \dots, t-1\}$).

Definition 2.7 (Decision-CRT_LWR). Let $n, m \geq 1$, and let $t, q \geq 2$ be coprime. Let $\mathbf{s} \in \mathbb{Z}_{tq}^n$ be a secret vector and $S_R \subset \mathbb{Z}_t$ non-empty. Choose $\mathbf{A} \in \mathbb{Z}_{tq}^{m \times n}$ uniformly. Compute

$$\mathbf{b}_t = \lfloor (\mathbf{A}\mathbf{s} \bmod t) \rfloor_{S_R} \in \mathbb{Z}_t^m,$$

$$\mathbf{b}_q = \left(\mathbf{A}\mathbf{s} + (\mathbf{b}_t - (\mathbf{A}\mathbf{s} \bmod t)) \bmod^{\pm t} \right) \bmod q \in \mathbb{Z}_q^m,$$

where the reduction $\bmod^{\pm t}$ is applied componentwise. The triple $(\mathbf{A}, \mathbf{b}_t, \mathbf{b}_q)$ is called a CRT_LWR sample. The decision problem asks to distinguish such samples from those generated by a uniformly random $\mathbf{u} \in \mathbb{Z}_{tq}^m$ with $(\mathbf{b}_t, \mathbf{b}_q) = (\lfloor \mathbf{u} \bmod t \rfloor_{S_R}, \mathbf{u} \bmod q)$.

2.7 Reduction from CRT_LWR to LWR

We first recall the standard LWR problem. For moduli $N > p$, a secret $\mathbf{s} \in \mathbb{Z}_N^n$, and a random $\mathbf{A} \in \mathbb{Z}_N^{m \times n}$, define $\mathbf{v} = \mathbf{A}\mathbf{s} \bmod N$. The LWR sample is $(\mathbf{A}, \mathbf{b})$ with $\mathbf{b} = \lfloor \frac{p}{N}\mathbf{v} \rfloor \bmod p$ (downward rounding). This variant is equivalent to the usual nearest-integer LWR.

Set $N = tq$ and $p = q$, so $\frac{p}{N} = 1/t$. Write $\mathbf{v} = t\mathbf{L}_1 + \mathbf{L}_2$ with $0 \leq \mathbf{L}_1 < q$, $0 \leq \mathbf{L}_2 < t$. Then the LWR sample is $(\mathbf{A}, \mathbf{b} = \mathbf{L}_1 \bmod q)$.

Theorem 2.1 (Singleton rounding set). *Let $t, q \geq 2$ be coprime, $c = \lceil \frac{t-1}{2} \rceil$, and $S_R = \{c\}$. Any distinguisher for Decision-CRT_LWR with these parameters also distinguishes downward rounding LWR with modulus tq and rounding modulus q , with the same advantage.*

Proof (Sketch). Given an LWR sample $(\mathbf{A}, \mathbf{b})$, construct $\mathbf{b}_t = c$ and $\mathbf{b}_q = t\mathbf{b} + c \bmod q$. If $\mathbf{b}$ is genuine, $\mathbf{v} = t\mathbf{b} + \mathbf{L}_2$ and $c - \mathbf{L}_2 \in \text{Sym}(t)$, so $\mathbf{b}_q = \mathbf{v} + (c - \mathbf{L}_2) \bmod q = t\mathbf{b} + c \bmod q$ matches the CRT_LWR definition. If $\mathbf{b}$ is uniform, $\mathbf{b}_q$ is uniform (since t invertible mod q) and $\mathbf{b}_t$ constant c , matching the random CRT_LWR distribution. Hence any CRT_LWR distinguisher directly distinguishes LWR. $\square$

Theorem 2.2 (Two-element rounding set). *Let $t \geq 2$ be even, q coprime to t , $c = \lfloor \frac{t-2}{4} \rfloor$, and $S_R = \{c, c + \frac{t}{2}\}$. Any distinguisher for Decision-CRT_LWR with these parameters also distinguishes downward rounding LWR with modulus tq and rounding modulus $2q$, with the same advantage.*

Proof (Sketch). From a downward rounding LWR sample $(\mathbf{A}, \mathbf{b})$ (modulus $2q$), write $\mathbf{b} = 2\mathbf{L}_1 + \boldsymbol{\delta}$ with $\boldsymbol{\delta} \in \{0, 1\}^m$. Construct $\mathbf{b}_t = c + \boldsymbol{\delta} \frac{t}{2}$ and $\mathbf{b}_q = \frac{t}{2}\mathbf{b} + c \bmod q$. For a genuine sample, $\mathbf{v} = t\mathbf{L}_1 + \mathbf{L}_2$, and one checks that $\lfloor \mathbf{L}_2 \rfloor_{S_R} = c + \boldsymbol{\delta} \frac{t}{2}$ and $\mathbf{b}_q = \mathbf{v} + (\lfloor \mathbf{L}_2 \rfloor_{S_R} - \mathbf{L}_2) \bmod q = t\mathbf{L}_1 + c + \boldsymbol{\delta} \frac{t}{2} = \frac{t}{2}\mathbf{b} + c$, matching the CRT_LWR construction. If $\mathbf{b}$ is uniform, the map $\mathbf{b} \mapsto (\boldsymbol{\delta}, \frac{t}{2}\mathbf{b} + c)$ is bijective, so $(\mathbf{b}_t, \mathbf{b}_q)$ is uniform over $\{c, c + \frac{t}{2}\}^m \times \mathbb{Z}_q^m$, exactly the random CRT_LWR distribution. Thus a CRT_LWR distinguisher yields an LWR distinguisher. $\square$

The specific value of c is not essential; any $c_0 \in \mathbb{Z}_t$ gives an equivalent problem via a translation of the secret. Moreover, the downward rounding variant is equivalent to the standard nearest-integer rounding LWR via a simple transformation (e.g., adjusting the secret or adding a constant), so the reductions apply to the usual LWR as well.

2.8 CRT_MLWR Problem

We extend the definition to the module setting.

Definition 2.8 (Decision-CRT_MLWR _{n,t,q,k,l,χ}). Let n, t, q, k, l be positive integers with t and q coprime, and let χ be a distribution over $\mathcal{R}_{tq}$. Let $S_R \subset \mathbb{Z}_t$ be a non-empty set. For a secret $\mathbf{s} \in \mathcal{R}_{tq}^l$, choose $\mathbf{A} \leftarrow \mathcal{R}_{tq}^{k \times l}$ uniformly and compute

$$\mathbf{b}_t = \lfloor (\mathbf{A}\mathbf{s} \bmod t) \rfloor_{S_R}, \quad \mathbf{b}_q = (\mathbf{A}\mathbf{s} + (\mathbf{b}_t - (\mathbf{A}\mathbf{s} \bmod t)) \bmod^{\pm t}) \bmod q,$$

where all operations are componentwise. The decision problem asks to distinguish $(\mathbf{A}, \mathbf{b}_t, \mathbf{b}_q)$ from $(\mathbf{A}, \lfloor \mathbf{b} \bmod t \rfloor_{S_R}, \mathbf{b} \bmod q)$ with $\mathbf{b} \leftarrow \mathcal{R}_{tq}^k$ uniform. Formally, the advantage of an adversary $\mathcal{A}$ is

$$\begin{aligned} \text{Adv}_{n,t,q,k,l,\chi}^{\text{CRT_MLWR}}(\mathcal{A}) = \\ \left| \Pr[\mathcal{A}(\mathbf{A}, \lfloor \mathbf{b} \bmod t \rfloor_{S_R}, \mathbf{b} \bmod q) = 1 \mid \mathbf{A} \leftarrow \mathcal{R}_{tq}^{k \times l}, \mathbf{b} \leftarrow \mathcal{R}_{tq}^k] \right. \\ \left. - \Pr[\mathcal{A}(\mathbf{A}, \mathbf{b}_t, \mathbf{b}_q) = 1 \mid \mathbf{A} \leftarrow \mathcal{R}_{tq}^{k \times l}, \mathbf{s} \leftarrow \chi^l] \right|. \end{aligned}$$

2.9 Public Key Encryption and Key Encapsulation Mechanism

We recall the standard definitions of PKE and KEM.

Definition 2.9 (PKE). A public-key encryption scheme consists of three PPT algorithms (KeyGen, Enc, Dec):

- KeyGen(1^λ): outputs a public key pk and a secret key sk .
- Enc(pk, μ): outputs a ciphertext c for message μ .
- Dec(sk, c): outputs a message μ (or $\perp$).

Definition 2.10 (KEM). A key encapsulation mechanism consists of three PPT algorithms (KeyGen, Encaps, Decaps):

- KeyGen(1^λ): outputs (pk, sk) .
- Encaps(pk): outputs a shared key K and a ciphertext c .
- Decaps(sk, c): outputs a shared key K .

Definition 2.11 (Completeness/DFR). A PKE (or KEM) is $(1 - \delta)$ -correct if for all (pk, sk) generated by KeyGen and any message μ (or any randomness of Encaps),

$$\Pr[\text{Dec}(sk, \text{Enc}(pk, \mu)) \neq \mu] \leq \delta,$$

where the probability is over the encryption randomness. The quantity δ is called the decryption failure rate (DFR). In the (Q)ROM, the probability also includes the randomness of the (quantum) random oracle.

Definition 2.12 (IND-CPA security of PKE). For an adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, define

$$\text{Adv}_{\text{PKE}}^{\text{IND-CPA}}(\mathcal{A}) = \left| \Pr \left[b' = b \mid \begin{array}{l} (pk, sk) \leftarrow \text{KeyGen}(1^\lambda) \\ (\mu_0, \mu_1, st) \leftarrow \mathcal{A}_1(pk) \\ b \leftarrow \{0, 1\}; c \leftarrow \text{Enc}(pk, \mu_b) \\ b' \leftarrow \mathcal{A}_2(pk, c, st) \end{array} \right] - \frac{1}{2} \right|.$$

Definition 2.13 (IND-CPA and IND-CCA2 security of KEM). For an adversary $\mathcal{A}$,

$$\text{Adv}_{\text{KEM}}^{\text{IND-CPA}}(\mathcal{A}) = \left| \Pr \left[b' = b \mid \begin{array}{l} (pk, sk) \leftarrow \text{KeyGen}(1^\lambda) \\ b \leftarrow \{0, 1\}; (K_0, c) \leftarrow \text{Encaps}(pk) \\ K_1 \leftarrow \mathcal{K}; b' \leftarrow \mathcal{A}(pk, c, K_b) \end{array} \right] - \frac{1}{2} \right|,$$

where $\mathcal{K}$ is the key space. For IND-CCA2, $\mathcal{A}$ additionally has access to a decapsulation oracle $\text{Decaps}(sk, \cdot)$ at any time (both before and after receiving the challenge ciphertext), but may not query the challenge ciphertext itself.

Definition 2.14 ((Q)ROM security). Let $\mathcal{S} \in \{\text{PKE}, \text{KEM}\}$. For $T, \epsilon > 0$, $\mathcal{S}$ is said to be (T, ϵ) -ATK secure in the (Q)ROM if for any (quantum) adversary $\mathcal{A}$ running in time at most T , making at most q_H (quantum) random oracle queries and (if applicable) at most q_D decryption/decapsulation queries, it holds that $\text{Adv}_{\mathcal{S}}^{\text{ATK}}(\mathcal{A}) < \epsilon$.

2.10 Encoding

In standard LPR encryption, each plaintext bit is directly embedded into a polynomial coefficient by multiplying by $\lfloor tq/2 \rfloor$. To achieve a decryption failure rate strictly below the target security bound $2^{-\lambda}$ without inflating the dimension n or the modulus tq , we incorporate error-correcting codes (ECC) into the plaintext encoding. The choice of encoding depends on the security level and the ratio between the polynomial dimension n and the plaintext length $\kappa = \lambda$ (the message space of the KEM is consistent with the security level):

- **Repetition code (128-bit security):** Let $k_{\text{vec}} = n/\kappa$ (an integer). Each message bit is repeated k_{vec} times across the polynomial coefficients. During decryption, the receiver sums the k_{vec} recovered integer values (each in $[-tq/2, tq/2]$) corresponding to the same message bit, obtaining a total S . If $|S| < k_{\text{vec}} \cdot tq/4$, the bit is decoded as 0; otherwise as 1.
- **BCH code (256-, 384-, and 512-bit security):** When n is close to κ , repetition becomes inefficient. Instead, we use a shortened binary BCH code that maps the κ -bit message to an n -bit codeword. The decoder can correct up to τ symbol errors (in the recovered integer values after hard thresholding), reducing the effective DFR to the required negligible level while maximizing bandwidth utilization.

The encoding step is integrated into the Encode function used in Algorithm 11. Concretely, Encode first applies the chosen ECC to expand the κ -bit message into an n -bit string, and then maps each bit to a coefficient $\lfloor tq/2 \rfloor$ as described in Algorithm 8. The exact parameters (k_{vec} , BCH parameters) are given in Table 2.

3 PKE and KEM Schemes

3.1 Overview

Our design builds upon the well-established LPR framework but introduces two novel techniques—**Variable Modulus** and **CRT Compression**—that together offer greater flexibility in parameter selection and enable compact bandwidth. In this section, we first outline the foundational concepts and the challenges they pose, and then provide a detailed explanation of Lore’s core innovations.

Foundation: LWE-based PKE and the FO Transform. The Learning With Errors (LWE) problem and its structured variants (Ring/Module-LWE) [6, 30] provide the foundation for modern lattice-based cryptography. In a canonical LWE encryption scheme, the public key is $(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \bmod q)$ and a message μ is encrypted as $(\mathbf{C}_u = \mathbf{A}^\top \mathbf{s}' + \mathbf{e}' \bmod q, C_v = \mathbf{b}^\top \mathbf{s}' + e'' + \lfloor \frac{q}{2} \rfloor \cdot \mu \bmod q)$. Decryption correctness relies on the magnitude of an error term $E = \mathbf{e}^\top \mathbf{s}' - \mathbf{e}'^\top \mathbf{s} + e''$ being less than $q/4$.

To achieve IND-CCA security, these IND-CPA secure PKEs are typically transformed into KEMs using the Fujisaki–Okamoto (FO) transform [21]. A critical requirement stems from the concrete security bound of an FO-derived KEM: as formalized in [25], this bound inevitably includes an additive term equal to the decryption error probability of the base PKE. Consequently, the underlying PKE must have a DFR that is not only negligible, but specifically matches the target security parameter λ (i.e., $\text{DFR} < 2^{-\lambda}$). A non-negligible DFR directly undermines the security proof and can expose the scheme to failure exploitation attacks. While generic compilers can transform a PKE with a non-negligible DFR into one with a negligible DFR [15], they do so at a significant cost to efficiency and bandwidth—for instance, by expanding the ciphertext size multiplicatively.

Despite these stringent theoretical requirements, many lattice-based KEMs exhibit DFRs that are not optimally aligned with their claimed security levels. For instance, Kyber [6]—one of the NIST-selected algorithms—employs a fixed modulus across all security levels to simplify implementation. This design choice, while practical, results in a DFR of approximately 2^{-174} for Kyber1024, which is significantly higher than the 2^{-256} threshold required for 256-bit security. This gap shows that the fixed-modulus constraint prevents achieving an optimally balanced parameter set for higher security levels. Similar trade-offs can

be observed in other schemes like SMAUG-T [14]. This inflexibility—the inability to fine-tune the DFR to precisely match each security level—is the primary challenge our work addresses.

Core Innovation: Variable Modulus and CRT Compression. To overcome this limitation, we introduce **Variable Modulus**. Instead of using a single modulus q , Lore operates with modulus tq , where $q = 257$ is fixed and t is a small power-of-two (2 or 4) chosen per security level. This decouples the modulus size from the lattice dimension, enabling fine-grained control over the error term E and allowing us to achieve a DFR strictly below $2^{-\lambda}$ without over-parameterization.

To implement variable modulus efficiently, we propose **CRT Compression**. Since t and q are coprime, every element $x \in \mathbb{Z}_{tq}$ is uniquely represented by its residues $(x_t, x_q) \in \mathbb{Z}_t \times \mathbb{Z}_q$ via the Chinese Remainder Theorem. All computations can then be performed on these smaller rings, and transmission can store only the residues, reducing bandwidth. Inspired by LWR, we apply a controlled rounding step to the CRT components: rounding x_t to a small set $S_R \subset \mathbb{Z}_t$ (e.g., a singleton or two elements) reduces its storage to $\log_2 |S_R|$ bits, while rounding x_q to a coarser set further compresses C_v . This rounding introduces a small, predictable error, analogous to LWR, and its security is reducible to standard LWR. Together, Variable Modulus and CRT Compression enable Lore to achieve both low DFR and compact bandwidth.

Optimizations for Compactness and Efficiency. To further enhance practicality, Lore incorporates several optimizations:

- **Fixed Weight Sampling:** The secret vectors in Lore (e.g., $\mathbf{s}, \mathbf{s}'$) are sampled from a fixed-weight distribution: the total number of coefficients equal to each value in $\{-2, -1, 0, 1, 2\}$ is predetermined (see Table 2). When restricted to $\{-1, 0, 1\}$, this reduces to the fixed Hamming weight distribution, which has been widely used in LWE-based designs [14, 27]. Fixed-weight sampling reduces DFR and improves computational efficiency, albeit at the cost of slightly reducing security by enabling hybrid attacks [9, 19, 36].
- **Efficient modulus choice:** To support fast polynomial multiplication, we need a primitive $2n$ -th root of unity in $\mathbb{Z}_q$. For a fully splitting NTT, this requires a prime q such that $2n \mid (q-1)$. Our fixed modulus $q = 257$ satisfies $q-1 = 256 = 2^8$, which only guarantees a 256-th root of unity. Hence, for $n = 512$ (requiring a 1024-th root), we cannot use a full NTT. Instead, we employ an incomplete NTT that factors $\mathcal{R}_q$ into irreducible polynomials of degree $d' = 2$ or 4 (as in [31]), achieving comparable efficiency to power-of-two NTTs. The parameter t is chosen as a small power-of-two (2 or 4), allowing highly efficient arithmetic in $\mathcal{R}_t$ without NTT.
- **Lossless Coding of Public Key and Ciphertext:** To further reduce bandwidth, we use a simple variable-length encoding for elements of $\mathbf{b} \in \mathbb{Z}_q$

(with $q = 257$). As shown in Algorithms 1 and 2, values $0 \leq x \leq 254$ are encoded as 8 bits, while $x = 255$ and $x = 256$ are encoded as 9 bits (prefix 255 followed by one bit). Since the elements are uniformly distributed, the average length is $\frac{2}{257} \cdot 9 + \frac{255}{257} \cdot 8 \approx 8.008$ bits per element, saving nearly 1 bit per element compared to a naive 9-bit encoding. This method extends naturally to ring elements by applying it coefficient-wise.

- **Error correction:** We incorporate error-correcting codes (ECC) into the plaintext encoding before encryption. Since the underlying LWE encryption is IND-CPA secure, adding ECC does not affect security, and constant-time implementations of repetition codes and BCH codes are well studied in lattice-based designs. So the ECC parameters are utilized to further lower the effective DFR as listed in Table 2 (e.g., repetition for 128-bit security, BCH for higher levels).

Algorithm 1 EncodeToByte

Input: Integer $x \in \mathbb{Z}_q$ where $q = 257$

Output: Bit string x'

- 1: **if** $x < 255$ **then**
 - 2: $x' \leftarrow$ 8-bit binary representation of x {Padded with leading zeros if necessary}
 - 3: **else**
 - 4: $x' \leftarrow 11111111 \parallel$ 1-bit binary representation of $(x - 255)$ { $11111111 = 255$ }
 - 5: **end if**
 - 6: **return** x'
-

Algorithm 2 DecodeFromByte

Input: Bit string x'

Output: Integer $x \in \mathbb{Z}_q$ where $q = 257$

- 1: Let $b_0b_1 \cdots b_7$ be the first 8 bits of x'
 - 2: **if** $b_0b_1 \cdots b_7 = 11111111$ **then**
 - 3: Let b_8 be the 9th bit of x' (must exist)
 - 4: $x \leftarrow 255 +$ value of b_8 { b_8 is 0 or 1}
 - 5: **else**
 - 6: $x \leftarrow$ integer value of $b_0b_1 \cdots b_7$
 - 7: **end if**
 - 8: **return** x
-

3.2 Core Strategy: Variable Modulus and CRT Compression

Variable Modulus. As discussed, the FO transform’s security proof requires that the DFR of the underlying PKE be strictly less than $2^{-\lambda}$. However, many practical schemes adopt a fixed modulus across all security levels for implementation simplicity, often at the cost of an optimally balanced parameter set that precisely meets this DFR requirement. To address this, we introduce the Variable Modulus technique, which employs a modulus of the form tq , where q is a fixed base and t is a security-level-dependent parameter. On one hand, this provides the flexibility to select parameters that yield a DFR just below $2^{-\lambda}$ without over-engineering. On the other hand, it simplifies implementation by reusing core arithmetic modules (e.g., the NTT in $\mathcal{R}_q$) across different security levels, thereby offering a new trade-off between implementation complexity and parameter flexibility. As will be detailed later, this flexibility also enables improved bandwidth efficiency across all security levels compared to previous fixed-modulus designs.

CRT Compression. Compression via rounding is widely used in lattice-based KEMs to reduce bandwidth (e.g., Kyber [6] and Saber [17]). The idea is to scale an element by a factor p/q and round to an integer, discarding low-order bits while introducing a small deterministic error. In Lore, elements are represented in CRT form: for coprime t and q , any $x \in \mathbb{Z}_{tq}$ corresponds uniquely to the pair (x_q, x_t) with $x_q = x \bmod q$ and $x_t = x \bmod t$. Storing the pair instead of x already reduces bandwidth when using the lossless coding for storing x_q combined with small t (e.g., $t = 2$ or 4).

Rounding in the CRT representation is facilitated by the fact that adding an integer ϵ to x adds the same ϵ to both residues: $x'_q = x_q + \epsilon \pmod{q}$, $x'_t = x_t + \epsilon \pmod{t}$. Thus, to round x to a nearby value $x' = x + \epsilon$, we can adjust both residues by the same ϵ . Our **CRT Compression** chooses ϵ so that one of the residues (say x_t) falls into a small predefined set $S_R \subset \mathbb{Z}_t$, thereby reducing its storage cost.

Take $t = 2$ as a concrete example. For a given x_t , set $\epsilon = 0 - x_t \bmod 2$ (i.e., $\epsilon = 0$ if $x_t = 0$, and $\epsilon = 1$ if $x_t = 1$). Then define $x'_q = x_q + \epsilon \pmod{q}$ and $x'_t = 0$. The new element $x' = x + \epsilon$ satisfies $x' \equiv x'_q \pmod{q}$ and $x' \equiv 0 \pmod{2}$; its CRT representation is $(x'_q, 0)$. Storing only x'_q (and implicitly knowing that the $\mathbb{Z}_2$ component is fixed to 0) saves 1 bit compared to storing the full pair. The error ϵ is deterministic and uniformly distributed in $\{0, 1\}$. This operation is analogous to rounding in LWR and yields a sample of the CRT-LWR problem defined in Section 2.6, whose security reduces to standard LWR (Section 2.7).

Different parts of the public key and ciphertext tolerate different amounts of rounding error. For instance, errors in $\mathbf{C}_u$ are multiplied by the secret during decryption, whereas those in C_v are not. Therefore, Lore applies two distinct compression strategies:

- **Public key and C_u :** Compress the $\mathbb{Z}_t$ component by rounding x_t to a set S_R . The size of S_R (hence the bits needed to encode the rounded value) depends on the security level: for our four target levels (128, 256, 384, 512 bits), this requires 0, 0, 1, 1 bits respectively. The rounding error is uniformly distributed in $\{0, 1\}$, and the compression and decompression procedures are given in Algorithms 3 and 5.
- C_v : Compress the $\mathbb{Z}_q$ component by rounding x_q to a set of size 2^l , where $l = 1$ for the four security levels. This yields a more aggressive bandwidth reduction. The rounding error is uniformly distributed over the integers in the interval $[-q/2^{l+1}, q/2^{l+1}]$. The corresponding algorithms are Algorithms 4 and 6.

All algorithms are designed to be applied componentwise to ring elements. And this flexible design allows us to tune the rounding error and bandwidth savings independently for each security level and for each part of the ciphertext.

Algorithm 3 Compression

Input: $x_q \in \mathbb{Z}_q, x_t \in \mathbb{Z}_t$;

Output: $x'_q \in \mathbb{Z}_q, x'_t \in \mathbb{Z}_{t/2}$;

- 1: **assert** $x_q = x \bmod q$ and $x_t = x \bmod t$ for some $x \in \mathbb{Z}_{tq}$
 - 2: $x'_t = \lfloor x_t/2 \rfloor$
 - 3: $x''_t = 2x'_t \bmod t$
 - 4: $x'_q = x_q + (x''_t - x_t \bmod^\pm t) \bmod q$
 - 5: **return** (x'_q, x'_t)
-

Algorithm 4 Compression $_{C_v}$

Input: $x_q \in \mathbb{Z}_q, x_t \in \mathbb{Z}_t, l$: target bit length for storing x'_q ;

Output: $x'_q \in \mathbb{Z}_{2^l}, x'_t \in \mathbb{Z}_t$;

- 1: **assert** $x_q = x \bmod q$ and $x_t = x \bmod t$ for some $x \in \mathbb{Z}_{tq}$
 - 2: $x'_q = \lfloor 2^l x_q / q + 1/2 \rfloor \bmod 2^l$
 - 3: $x''_q = \lfloor \frac{q}{2^l} \cdot x'_q + \frac{1}{2} \rfloor \bmod q$
 - 4: $x'_t = x_t + (x''_q - x_q \bmod^\pm q) \bmod t$
 - 5: **return** (x'_q, x'_t)
-

3.3 Public Key Encryption Algorithm

The formal definition of the PKE scheme is given below. We denote by $\mathcal{R}_{tq} = \mathbb{Z}_{tq}[x]/(x^n + 1)$ the polynomial ring, and by $\mathcal{R}_q, \mathcal{R}_t$ the respective residue rings.

Algorithm 5 Decompression

Input: $x'_q \in \mathbb{Z}_q, x'_t \in \mathbb{Z}_{t/2}$;**Output:** $x' \in \mathbb{Z}_{tq}$;

- 1: $x''_t = 2x'_t$
 - 2: $x' = (x'_q \cdot t \cdot (t^{-1} \bmod q) + x''_t \cdot q \cdot (q^{-1} \bmod t)) \bmod tq$
 - 3: **return** x' .
-

Algorithm 6 Decompression_{C_v}

Input: $x'_q \in \mathbb{Z}_{2^l}, x'_t \in \mathbb{Z}_t, l$;**Output:** $x' \in \mathbb{Z}_{tq}$;

- 1: $x''_q = \lfloor \frac{q}{2^l} \cdot x'_q + \frac{1}{2} \rfloor \bmod q$
 - 2: $x' = (x''_q \cdot t \cdot (t^{-1} \bmod q) + x'_t \cdot q \cdot (q^{-1} \bmod t)) \bmod tq$
 - 3: **return** x' .
-

All operations on vectors and matrices are applied componentwise to the polynomial coefficients.

Key Generation. The keygen process of our scheme follows a standard MLWR key generation process under the modulus tq as shown in Algorithm 7. The main difference is that CRT Compression is used instead of the rounding technology of LWR.

Algorithm 7 PKE_KeyGen

Input: 1^λ ;**Output:** pk, sk ;

- 1: $seed \leftarrow \{0, 1\}^{\rho_0}$
 - 2: $(seed_A, seed_{sk}) \leftarrow H_{gen}(seed)$
 - 3: $\mathbf{A} \leftarrow \text{expandA}(seed_A) \in \mathcal{R}_{tq}^{k \times k}$
 - 4: $\mathbf{s} \leftarrow \text{expandS}(\chi, seed_{sk}) \in \mathcal{R}_{tq}^k$
 - 5: $\mathbf{b}_q \leftarrow \mathbf{A}\mathbf{s} \bmod q, \mathbf{b}_t \leftarrow \mathbf{A}\mathbf{s} \bmod t$
 - 6: $(\mathbf{b}'_q, \mathbf{b}'_t) \leftarrow \text{Compression}(\mathbf{b}_q, \mathbf{b}_t)$
 - 7: $\mathbf{b}''_q \leftarrow \text{EncodeToByte}(\mathbf{b}'_q)$
 - 8: **return** $pk = (seed_A, \mathbf{b}''_q, \mathbf{b}'_t), sk = \mathbf{s}$.
-

Encryption. The encryption procedure is given in Algorithm 11. During encryption, all elements are kept in CRT representation, so computations are performed separately modulo q and modulo t . Line 6 is equivalent to first decompressing $\mathbf{b}'$ via $\text{Decompress}(\mathbf{b}'_q, \mathbf{b}'_t)$ and then computing (C'_{vq}, C'_{vt}) as $\text{Compression}_{C_v}(\mathbf{b}'^\top \mathbf{s}' + \text{Encode}(\mu, n, \kappa, \lambda) + e'' \bmod q, \mathbf{b}'^\top \mathbf{s}' + \text{Encode}(\mu, n, \kappa, \lambda) + e'' \bmod t)$. The extra noise term e'' ensures that $C_v \bmod t$ is uniformly distributed over $\mathbb{Z}_t$.

Algorithm 8 Encode

Input: $\mu \in \{0, 1\}^\kappa$, Polynomial degree n , Message length κ , Security level λ ;

Output: $M \in \mathcal{R}_{tq}$;

```
1: if  $\lambda = 128$  then  
2:    $M \leftarrow \text{Encode\_REP}(\mu, n, \kappa)$   
3: else  
4:    $M \leftarrow \text{Encode\_BCH}(\mu, n, \kappa)$   
5: end if  
6: return  $M$ 
```

Algorithm 9 Encode.REP

Input: $\mu \in \{0, 1\}^\kappa$, n , κ ;

Output: $M \in \mathcal{R}_{tq}$;

```
1:  $k_{vec} \leftarrow n/\kappa$   
2: for  $i$  from 0 to  $\kappa - 1$  do  
3:   for  $j$  from 0 to  $k_{vec} - 1$  do  
4:      $idx \leftarrow i \cdot k_{vec} + j$   
5:      $M_{idx} \leftarrow \mu_i \cdot \frac{tq}{2}$   
6:   end for  
7: end for  
8: return  $M$ 
```

Algorithm 10 Encode.BCH

Input: $\mu \in \{0, 1\}^\kappa$, n , κ ;

Output: $M \in \mathcal{R}_{tq}$;

```
1:  $c \leftarrow \text{BCH.Encode}(\mu, n, \kappa)$   $\{c \in \{0, 1\}^n\}$   
2: for  $i$  from 0 to  $n - 1$  do  
3:    $M_i \leftarrow c_i \cdot \frac{tq}{2}$   
4: end for  
5: return  $M$ 
```

Decryption. The decryption process is given in Algorithm 15. Steps 2–4 are equivalent to first decompressing $\mathbf{C}_u$ and C_v via Decompression and Decompression $_{C_v}$, then computing $res = C_v - \mathbf{C}_u^\top \mathbf{s} \bmod tq$.

3.4 Key Encapsulation Mechanism Algorithm

We now present the Lore KEM scheme, which is obtained by applying the Fujisaki–Okamoto transform [21] to the IND-CPA-secure PKE described in Section 3.3. The resulting KEM is IND-CCA2-secure in the (Q)ROM under the assumptions stated in Section 4. Algorithms 16, 17, and 18 define key generation, encapsulation, and decapsulation, respectively.

Algorithm 11 PKE_Encryption

Input: $pk, \mu \in \{0, 1\}^\kappa, seed_r$;**Output:** $\mathbf{C}$;

- 1: $\mathbf{A} \leftarrow \text{expandA}(seed_A) \in \mathcal{R}_{tq}^{k \times k}$
 - 2: $\mathbf{s}' \leftarrow \text{expandS}(\chi, seed_r) \in \mathcal{R}_{tq}^k, e'' \leftarrow \text{expandS}(U(-\frac{t}{2}, \frac{t}{2}], seed_r) \in \mathcal{R}_{tq}$
 - 3: $\mathbf{b}_q' \leftarrow \text{DecodeFromByte}(\mathbf{b}_q'')$
 - 4: $\mathbf{C}_{uq} \leftarrow \mathbf{A}^\top \mathbf{s}' \bmod q, \mathbf{C}_{ut} \leftarrow \mathbf{A}^\top \mathbf{s}' \bmod t$
 - 5: $(\mathbf{C}_{uq}', \mathbf{C}_{ut}') \leftarrow \text{Compression}(\mathbf{C}_{uq}, \mathbf{C}_{ut})$
 - 6: $C_{vq} \leftarrow \mathbf{b}_q'^\top \mathbf{s}' + \text{Encode}(\mu, n, \kappa, \lambda) + e'' \bmod q, C_{vt} \leftarrow (2\mathbf{b}_t')^\top \mathbf{s}' + \text{Encode}(\mu, n, \kappa, \lambda) + e'' \bmod t$
 - 7: $(C_{vq}', C_{vt}') \leftarrow \text{Compression}_{C_v}(C_{vq}, C_{vt}, l)$
 - 8: $\mathbf{C}_{uq}'' \leftarrow \text{EncodeToByte}(\mathbf{C}_{uq}')$
 - 9: **Output:** $\mathbf{C} \leftarrow (\mathbf{C}_{uq}'', \mathbf{C}_{ut}', C_{vq}', C_{vt}')$.
-

Algorithm 12 Decode

Input: $M \in \mathcal{R}_{tq}$, Polynomial degree n , Message length κ , Security level λ ;**Output:** Plaintext $\mu \in \{0, 1\}^\kappa$;

- 1: **if** $\lambda = 128$ **then**
 - 2: $\mu \leftarrow \text{Decode_REP}(M, n, \kappa)$
 - 3: **else**
 - 4: $\mu \leftarrow \text{Decode_BCH}(M, n, \kappa)$
 - 5: **end if**
 - 6: **return** μ
-

Algorithm 13 Decode_REP

Input: $M \in \mathcal{R}_{tq}, n, \kappa$;**Output:** Plaintext $\mu \in \{0, 1\}^\kappa$;

- 1: $k_{vec} \leftarrow n/\kappa$
 - 2: **for** i from 0 to $\kappa - 1$ **do**
 - 3: $S_0 \leftarrow 0, S_1 \leftarrow 0$
 - 4: **for** j from 0 to $k_{vec} - 1$ **do**
 - 5: $idx \leftarrow i \cdot k_{vec} + j$
 - 6: $S_0 \leftarrow S_0 + \text{abs}(M_{idx} \bmod \pm tq)$
 - 7: $S_1 \leftarrow S_1 + \text{abs}((tq/2 - M_{idx}) \bmod \pm tq)$
 - 8: **end for**
 - 9: **if** $S_0 > S_1$ **then**
 - 10: $\mu_i \leftarrow 1$
 - 11: **else**
 - 12: $\mu_i \leftarrow 0$
 - 13: **end if**
 - 14: **end for**
 - 15: **return** μ
-

Algorithm 14 Decode_BCH

Input: $M \in \mathcal{R}_{tq}, n, \kappa$;**Output:** Plaintext $\mu \in \{0, 1\}^\kappa$;

```
1: Let  $c'$  be an empty bit array of length  $n$ 
2: for  $i$  from 0 to  $n - 1$  do
3:   if  $4 \cdot |(M_i \bmod tq) - \frac{tq}{2}| < tq$  then
4:      $c'_i \leftarrow 1$ 
5:   else
6:      $c'_i \leftarrow 0$ 
7:   end if
8: end for
9:  $\mu \leftarrow \text{BCH\_Decode}(c', n, \kappa)$ 
10: return  $\mu$ 
```

Algorithm 15 PKE_Decryption

Input: sk, C ;**Output:** μ' ;

```
1:  $C'_{uq} \leftarrow \text{DecodeFromByte}(C''_{uq})$ 
2:  $res_q \leftarrow \lfloor \frac{q}{2^t} C'_{vq} + \frac{1}{2} \rfloor - C'_{uq}^\top \mathbf{s} \bmod q$ 
3:  $res_t \leftarrow C'_{vt} - (2C'_{ut})^\top \mathbf{s} \bmod t$ 
4:  $res \leftarrow (res_q \cdot t \cdot (t^{-1} \bmod q) + res_t \cdot q \cdot (q^{-1} \bmod t)) \bmod tq$ 
5:  $\mu' \leftarrow \text{Decode}(res, n, \kappa, \lambda)$ 
6: Output:  $\mu'$ .
```

Algorithm 16 KEM_KeyGen

Input: 1^λ ;**Output:** sk, pk ;

```
1:  $z \leftarrow \{0, 1\}^{256}$ 
2:  $(sk', pk) \leftarrow \text{PKE\_KeyGen}(1^\lambda)$ 
3: return  $(sk = (sk', pk, H(pk), z), pk)$ 
```

Algorithm 17 KEM_Encaps

Input: pk ;**Output:** c, K ;

```
1:  $m \leftarrow \{0, 1\}^\lambda$ 
2:  $(\bar{K}, r) \leftarrow G(H(m) || H(pk))$ 
3:  $c \leftarrow \text{PKE\_Encryption}(pk, m, r)$ 
4:  $K \leftarrow \text{KDF}(\bar{K} || H(c))$ 
5: return  $(c, K)$ 
```

3.5 Specification

The Lore PKE and KEM schemes rely on the following building blocks, which are instantiated with standard symmetric primitives:

Algorithm 18 KEM.Decaps

Input: sk, c ;**Output:** K ;

```
1:  $m' \leftarrow \text{PKE\_Decryption}(sk, c)$ 
2:  $(\bar{K}', r') \leftarrow G(H(m') || H(pk))$ 
3:  $c' \leftarrow \text{PKE\_Encryption}(pk, m', r')$ 
4: if  $c' = c$  then
5:    $K \leftarrow \text{KDF}(\bar{K}' || H(c'))$ 
6: else
7:    $K \leftarrow \text{KDF}(z || H(c))$ 
8: end if
9: return  $K$ 
```

- **Hash function** H_{gen} : Used to split a master seed into seeds for matrix generation and secret sampling. Implemented with SHAKE256/SM3, producing the required number of output bits.
- **Extendable output function** `expandA`: Derives the public matrix $\mathbf{A} \in \mathcal{R}_{tq}^{k \times k}$ from a seed $seed_A$. It uses SHAKE256/SM3 and the output is interpreted as integers in $[0, tq - 1]$.
- **Extendable output function** `expandS`: Generates secret vectors (e.g., $\mathbf{s}$, $\mathbf{s}'$, e'') from a seed according to the fixed-weight distribution described in Table 2 or uniform distribution on $\mathbb{Z}_t$. It employs SHAKE256/SM3 to produce a stream of random bits, from which the positions and values of the non-zero coefficients are selected using a constant-time algorithm (e.g., based on rejection sampling or a fixed-weight encoding).

All hash functions (H , G , KDF) are instantiated using SHAKE256/SM3. In the KEM, H is used to hash the public key and the ciphertext; G is a hash function with output length $|\bar{K}| + |r|$, where $|r|$ denotes the number of random bits required by `PKE_Encryption`; and KDF is a key derivation function also implemented with SHAKE256/SM3. Note that both SHAKE and SM3 provide up to 256-bit security, and currently no hash function can achieve 384-bit or 512-bit security. Therefore, for implementations targeting those higher security levels, we currently use the 256-bit versions as temporary references. Once an algorithm that meets the required security level becomes available, it will be replaced accordingly.

4 Security Analysis

4.1 Completeness and DFR

With the algorithms described in Section 3.3, we first establish the correctness (completeness) of the Lore PKE scheme.

Theorem 4.1 (Completeness of Lore.PKE). Let $\mathbf{A} \in \mathcal{R}_{tq}^{k \times k}$, $\mathbf{b} \in \mathcal{R}_{tq}^k$, $e'' \in \mathcal{R}_{tq}$, and let the secret vectors $\mathbf{s}, \mathbf{s}' \in \mathcal{R}_{tq}^k$ be sampled from the fixed-weight distribution specified in Table 2. Assume that t and q are coprime. Define the errors introduced by the CRT compression steps (see Section 3.2) as

$$\begin{aligned}\epsilon_1 &:= \text{Decompress}(\mathbf{b}'_q, \mathbf{b}'_t) - \mathbf{A}\mathbf{s} \pmod{tq}, \\ \epsilon_2 &:= \text{Decompress}(\mathbf{C}'_{uq}, \mathbf{C}'_{ut}) - \mathbf{A}^\top \mathbf{s}' \pmod{tq}, \\ \epsilon_3 &:= \text{Decompression}_{C_v}(C'_{vq}, C'_{vt}, l) - (\mathbf{b}'^\top \mathbf{s}' + \text{Encode}(\mu, \kappa, \lambda) + e'') \pmod{tq},\end{aligned}$$

where $\mathbf{b}' = \text{Decompress}(\mathbf{b}'_q, \mathbf{b}'_t)$ is the decompressed public key. Let δ denote the Decryption Failure Rate (DFR), then for different error-correcting code (ECC) strategies, we have

$$\delta = \begin{cases} \kappa \cdot p_{vec}, & \text{repetition strategy,} \\ \sum_{j=\tau_{bch}+1}^{N_{cw}} \binom{N_{cw}}{j} p^j (1-p)^{N_{cw}-j}, & \text{BCH strategy.} \end{cases}$$

where the probability is taken over the randomness of the encryption algorithm. Then Lore.PKE is $(1 - \delta)$ -correct.

Proof. We first note that the encryption and decryption procedures are equivalent to a standard LWE encryption with modulus tq , except that the usual Gaussian noise is replaced by the deterministic rounding errors $\epsilon_1, \epsilon_2, \epsilon_3$ coming from CRT compression. Indeed, during decryption we compute

$$\begin{aligned}C_v - \mathbf{C}_u^\top \mathbf{s} &\equiv \text{Decompression}_{C_v}(C'_{vq}, C'_{vt}, l) - \text{Decompress}(\mathbf{C}'_{uq}, \mathbf{C}'_{ut})^\top \mathbf{s} \pmod{tq} \\ &\equiv (\mathbf{b}'^\top \mathbf{s}' + \text{Encode}(\mu, \kappa, \lambda) + e'' + \epsilon_3) - ((\mathbf{A}^\top \mathbf{s}' + \epsilon_2)^\top \mathbf{s}) \pmod{tq} \\ &\equiv \text{Encode}(\mu, \kappa, \lambda) + (\mathbf{A}\mathbf{s} + \epsilon_1)^\top \mathbf{s}' + e'' + \epsilon_3 - (\mathbf{A}\mathbf{s})^\top \mathbf{s}' - \epsilon_2^\top \mathbf{s} \pmod{tq} \\ &\equiv \text{Encode}(\mu, \kappa, \lambda) + \epsilon_1^\top \mathbf{s}' + e'' + \epsilon_3 - \epsilon_2^\top \mathbf{s} \pmod{tq}.\end{aligned}$$

The message μ is correctly recovered if the absolute value of the combined error term $E = \epsilon_1^\top \mathbf{s}' + e'' + \epsilon_3 - \epsilon_2^\top \mathbf{s}$ for each coefficient is strictly less than $tq/4$. Let $p = \Pr[|E| \geq tq/4]$ denote the raw failure probability of a single coefficient. To tightly bound the overall Decryption Failure Rate (DFR) δ , we evaluate it according to the chosen error-correcting code strategy:

- **Repetition Strategy (Soft Decision):** For a message of length κ , each bit is expanded to $k_{vec} = n/\kappa$ coefficients. The decoder aggregates the absolute distance of the errors across the k_{vec} coefficients. Let $E_{sum} = \sum_{j=1}^{k_{vec}} |E^{(j)}|$ be the convolution of k_{vec} independent absolute error distributions. A message bit decoding fails if $E_{sum} \geq k_{vec} \cdot \frac{tq}{4}$. Let this block failure probability be $p_{vec} = \Pr[E_{sum} \geq k_{vec} \frac{tq}{4}]$. By the union bound, the overall DFR is strictly bounded by:

$$\delta \leq \kappa \cdot p_{vec}$$

- **BCH Strategy (Hard Decision):** For a BCH code designed to correct up to τ_{bch} errors over an effective codeword length N_{cw} (where N_{cw} represents either the truncated standard code length or the shortened full polynomial dimension n), the decoder performs a hard decision on each coefficient. Since each of the N_{cw} coefficients fails independently with probability p , the overall DFR follows the tail of the binomial distribution:

$$\delta = \sum_{j=\tau_{bch}+1}^{N_{cw}} \binom{N_{cw}}{j} p^j (1-p)^{N_{cw}-j}$$

By choosing appropriate parameters t and ECC schemes, we ensure $\delta < 2^{-\kappa} = 2^{-\lambda}$, which proves the theorem. $\square$

Theorem 4.2 (Completeness of Lore.KEM). *Under the same notations and assumptions as in Theorem 4.1, Lore.KEM is also $(1 - \delta)$ -correct. That is, for every key pair (pk, sk) generated by $\text{PKE_KeyGen}(1^\lambda)$, the shared keys K (output by Encaps) and K' (output by Decaps) are identical with probability at least $1 - \delta$.*

Proof. The decapsulation algorithm first recovers m' via PKE_Decryption . If decryption succeeds (i.e., $m' = m$), then the recomputed ciphertext c' equals the original c , and consequently the derived keys satisfy $K = K'$. The probability of decryption failure is exactly δ by Theorem 4.1, hence the KEM is $(1 - \delta)$ -correct. No additional assumption on the hash functions is needed for correctness. $\square$

4.2 Security Proof

We employ the Fujisaki-Okamoto transformation in the (Q)ROM to construct KEMs; therefore, the security proof of our KEM relies on the IND-CPA security of the underlying PKE. In the ROM, Lore.KEM has a tight reduction from the IND-CPA security of PKE. Due to the non-negligible decryption failure rate, the standard FO reduction in the QROM is either inapplicable or loose. Thus, we prove the IND-CCA security of Lore.KEM based on the non-tight QROM reduction by proving the IND-CPA security of Lore.PKE, which aligns with that employed in SMAUG-T [14].

Theorem 4.3 (IND-CPA security of Lore.PKE). *Assume that H_{gen} is a pseudorandom function, and that expandA , expandS are pseudorandom generators. Then for any adversary $\mathcal{A}$ against the IND-CPA security of Lore.PKE, there exist adversaries $\mathcal{B}_0, \mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3, \mathcal{B}_4$ such that*

$$\begin{aligned} \text{Adv}_{\text{Lore.PKE}}^{\text{IND-CPA}}(\mathcal{A}) &\leq \text{Adv}_{H_{gen}}^{\text{PRF}}(\mathcal{B}_0) + \text{Adv}_{\text{expandA}, \text{expandS}}^{\text{PRG}}(\mathcal{B}_1) \\ &\quad + \text{Adv}_{n,t,q,k,k,\chi}^{\text{CRT_MLWR}}(\mathcal{B}_2) + \text{Adv}_{n,t,q,k,k,\chi}^{\text{CRT_MLWR}}(\mathcal{B}_3) \\ &\quad + \text{Adv}_{n,q,1,k,\chi}^{\text{MLWE}}(\mathcal{B}_4), \end{aligned}$$

where the precise parameters for the CRT_MLWR and MLWE problems are as defined in Sections 2.

Proof. We prove the theorem via a sequence of hybrid games $G_0, G_1, \dots, G_5$. Let Adv_i denote the advantage of the adversary in game G_i .

Game G_0 : This is the real IND-CPA game for Lore.PKE as defined in Section 3.3. The challenger generates (pk, sk) using the genuine algorithms and returns the encryption of μ_b for a random bit b .

Game G_1 : Replace the function H_{gen} (used to derive seeds $seed_A, seed_{sk}$ from a master seed) by a truly random function. Any distinguisher between G_0 and G_1 directly yields a distinguisher for H_{gen} , so $|\text{Adv}_1 - \text{Adv}_0| \leq \text{Adv}_{H_{gen}}^{\text{PRF}}(\mathcal{B}_0)$.

Game G_2 : Replace the pseudorandom outputs of expandA and expandS by uniformly random elements. By the pseudorandomness of these generators, we have $|\text{Adv}_2 - \text{Adv}_1| \leq \text{Adv}_{\text{expandA}, \text{expandS}}^{\text{PRG}}(\mathcal{B}_1)$.

Game G_3 : In this game, the vector $\mathbf{b}'$ (the decompressed version of $\mathbf{b}$) is replaced by a uniformly random element of $\mathcal{R}_{tq}^k$, instead of being derived as $\mathbf{b}' = \text{Decompress}(\text{Compression}(\mathbf{As}))$. By the CRT_MLWR assumption (Definition 2.8), the tuple $(\mathbf{A}, \mathbf{b}_t, \mathbf{b}_q)$ is indistinguishable from random; after decompression, $\mathbf{b}'$ becomes uniform. Hence $|\text{Adv}_3 - \text{Adv}_2| \leq \text{Adv}_{n,t,q,k,k,\chi}^{\text{CRT_MLWR}}(\mathcal{B}_2)$.

Game G_4 : Here we replace the compressed ciphertext components $\mathbf{C}'_{uq}$ and $\mathbf{C}'_{ut}$ by uniformly random elements: $\mathbf{C}'_{uq} \leftarrow \mathcal{R}_q^k$, and $\mathbf{C}'_{ut}$ is sampled uniformly from the rounding set S_{R1} (the set used for public-key compression, see Section 3.2). Under the same CRT_MLWR assumption applied to $\mathbf{C}_u$, we obtain $|\text{Adv}_4 - \text{Adv}_3| \leq \text{Adv}_{n,t,q,k,k,\chi}^{\text{CRT_MLWR}}(\mathcal{B}_3)$.

Game G_5 : Finally, we replace the compressed C'_{vq} and C'_{vt} by uniformly random elements: $C'_{vq} \leftarrow \lfloor \mathcal{R}_q \rfloor_{S_{R2}}$ (where S_{R2} is the rounding set for C_v) and $C'_{vt} \leftarrow \mathcal{R}_t$. By the MLWE assumption (specifically, the term $\mathbf{b}'^\top \mathbf{s}' + e''$ is pseudo-random), the distribution of (C'_{vq}, C'_{vt}) is indistinguishable from uniform. Thus $|\text{Adv}_5 - \text{Adv}_4| \leq \text{Adv}_{n,q,1,k,\chi}^{\text{MLWE}}(\mathcal{B}_4)$.

In G_5 , the ciphertext components are all independent of the message μ_u , hence the adversary's advantage is zero. Chaining the bounds gives the desired inequality. $\square$

Theorem 4.4 (IND-CCA security of Lore.KEM [10]). *Let G and H be quantum-accessible random oracles, and assume that the deterministic PKE obtained from Lore.PKE by fixing randomness is ϵ -injective (i.e., for any public key, the probability that two distinct messages encrypt to the same ciphertext is at most ϵ). Then for any IND-CCA adversary $\mathcal{A}$ making at most Q_{Dec} decapsulation queries, Q_G queries to G , and Q_H queries to H , at quantum depth at most d_G and d_H respectively, there exist adversaries $\mathcal{B}_1$ (IND-CPA) and $\mathcal{B}_2$ (against*

finding a decryption-failing ciphertext) such that

$$\begin{aligned} \text{Adv}_{\text{Lore.KEM}}^{\text{IND-CCA}}(\mathcal{A}) \leq & 2\sqrt{(d_G + 2)(\text{Adv}_{\text{Lore.PKE}}^{\text{IND-CPA}}(\mathcal{B}_1) + 8(Q_G + 1)/|\mathcal{M}|)} \\ & + \text{Adv}_{\text{Lore.PKE}}^{\text{DFR}}(\mathcal{B}_2) + 4\sqrt{d_H Q_H / |\mathcal{M}|} + \epsilon, \end{aligned}$$

where $|\mathcal{M}|$ is the message space size.

The bound follows from the generic QROM security proof for the $FO^\perp$ transform given in [10]. The IND-CPA security of Lore.PKE (Theorem 4.3) can be plugged into this bound to obtain concrete parameters.

4.3 Cost of known attacks

For the concrete security analysis, we list the best known lattice attacks and consider their costs for attacking our scheme. All the best known attacks rely on the Block–Korkine–Zolotarev (BKZ) lattice reduction algorithm [13, 24, 35]. The BKZ algorithm is a lattice basis reduction algorithm that repeatedly uses a Shortest Vector Problem (SVP) solver in small-dimensional projected sublattices. The dimension b of these projected sublattices is called the block-size. BKZ with block-size b hence relies on an SVP solver in dimension b . The block-size b governs both the runtime cost of BKZ and the quality of the reduced basis it produces. It provides a quality/time trade-off: If b gets larger, better quality will be guaranteed, but the time complexity for the SVP solver will exponentially increase. The time complexity of the b -BKZ algorithm is the same as the SVP solver for dimension b , up to polynomial factors. Hence the time complexity differs depending on the SVP solver used. The most efficient SVP algorithm uses the sieving method proposed by Becker et al. in [8] which takes time $\approx 2^{0.292b+o(b)}$. The fastest known quantum variant is proposed by Chailloux and Loyer in [12] and takes time $\approx 2^{0.257b+o(b)}$.

To evaluate the concrete hardness of the underlying MLWE problems, we utilize the Lattice Estimator library [2]. Unlike the simplified Core-SVP methodology [3] which typically ignores polynomial factors, we adopt the MATZOV cost model [38] for classical security (accounting for polynomial overheads) and the ChaLoy21 model [12] for quantum security (representing a conservative Quantum Core-SVP hardness) which correspond to the latest version of the Lattice Estimator library.

Even though the MLWR problems have some extra algebraic structure compared to the LWE problems, we do not currently know how to exploit it to improve the best known attacks. For this reason, we estimate the concrete hardness of the MLWR problems over the structured lattices as the concrete hardness of the corresponding LWE problems over the unstructured lattices.

We summarize the costs of the known attacks in Table 2. In the table, the required block-sizes for BKZ and the costs of the attacks are estimated using the

Lattice Estimator library [2] with the aforementioned models. The parameters for MLWE problem are chosen based on the above analysis.

Primal attack. Given an LWE instance $(\mathbf{A}, \mathbf{b}) \in \mathbb{Z}_q^{k \times l} \times \mathbb{Z}_q^k$, we first define the lattices $\Lambda_m = \{\mathbf{v} \in \mathbb{Z}^{l+m+1} : \mathbf{B}\mathbf{v} = \mathbf{0} \bmod q\}$ for all $m \leq k$, where $\mathbf{B} = (\mathbf{A}_{[m]} | \mathbf{Id}_m | \mathbf{b}_{[m]}) \in \mathbb{Z}_q^{m \times (l+m+1)}$, $\mathbf{A}_{[m]}$ is the uppermost $m \times l$ sub-matrix of $\mathbf{A}$ and $\mathbf{b}_{[m]}$ is the uppermost m -dimensional sub-vector of $\mathbf{b}$. As $(\mathbf{A}, \mathbf{b}) \in \mathbb{Z}_q^{k \times l} \times \mathbb{Z}_q^k$ is an LWE instance, there exist $\mathbf{s}$ and $\mathbf{e}$ short such that $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$. This implies that $(\mathbf{s} | \mathbf{e})$ is a short vector of Λ_m . The primal attack consists in running BKZ on Λ_m to find short vectors. The variable m is optimized to minimize the cost of the attack.

Dual attack. Given an LWE instance $(\mathbf{A}, \mathbf{b}) \in \mathbb{Z}_q^{k \times l} \times \mathbb{Z}_q^k$, we first define the lattices $\Lambda'_m = \{(\mathbf{u}, \mathbf{v}) \in \mathbb{Z}^m \times \mathbb{Z}^l : \mathbf{A}_{[m]}^\top \mathbf{u} - \mathbf{v} = \mathbf{0} \bmod q\}$ for all $m \leq k$, where $\mathbf{A}_{[m]}$ is the uppermost $m \times l$ sub-matrix of $\mathbf{A}$. If $(\mathbf{u}, \mathbf{v})$ is a short vector in Λ'_m , then $\mathbf{u}^\top \mathbf{b}_{[m]} = \mathbf{v}^\top \mathbf{s} + \mathbf{u}^\top \mathbf{e}_{[m]}$ is small if $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$ for short vectors $\mathbf{s}$ and $\mathbf{e}$, and is uniformly distributed modulo q if $\mathbf{b}$ is uniform and independent from $\mathbf{A}$ (here $\mathbf{e}_{[m]}$ refers to the uppermost m -dimensional sub-vector of $\mathbf{e}$). This provides a distinguishing attack. The dual attack consists in finding a short non-zero vector in the lattice Λ'_m using BKZ. The variable m is optimized to minimize the cost of the attack.

5 Performance

5.1 Parameter Sets

We propose four distinct parameter sets for the Lore PKE scheme, designed to meet the 128, 256, 384 and 512-bit classical security levels according to the requirements of NICCS [32]. All parameter sets are instantiated within the polynomial ring $\mathcal{R}_{tq} = \mathbb{Z}_{tq}[x] / \langle x^n + 1 \rangle$, with a fixed base modulus $q = 257$. The security-dependent factor t is chosen as a small power of two (2 or 4) to achieve the desired DFR while keeping the CRT representation efficient.

The cryptographic properties of the scheme are determined by the following choices, which are summarised in Table 2:

- **Fixed-Weight Secret Distribution.** The secret vectors $\mathbf{s}, \mathbf{s}' \in \mathcal{R}_{tq}^k$ are sampled from a distribution where the total number of coefficients equal to each value in $\{-2, -1, 0, 1, 2\}$ is fixed across all k polynomials. The exact counts for each security level are given in the notes of Table 2. This design reduces the DFR and improves efficiency, at the cost of a slight security loss that is accounted for in the SVP estimates.
- **CRT-Compressed Noise Distribution.** The cryptographic noise (e.g., the errors $\epsilon_1, \epsilon_2, \epsilon_3$ introduced in Theorem 4.1) arises from the rounding

steps of CRT Compression. Its distribution is fully determined by the parameters t (for the $\mathbb{Z}_t$ component) and l (for the $\mathbb{Z}_q$ component of C_v). This deterministic, discrete error is easy to analyse and allows for tight DFR bounds.

- **Bandwidth Computation.** The sizes of public key, ciphertext and secret key are computed as follows (all sizes are in bytes):

- **Public key:** 32 (for $seed_A$) plus $k \cdot n \cdot \frac{8.008 + (\log_2 t - 1)}{8}$, where 8.008 is the average number of bits needed to encode a $\mathbb{Z}_q$ element after the lossless coding of Algorithms 1 and 2, and $\log_2 t - 1$ is the size of the rounding set used for the $\mathbb{Z}_t$ component (see Section 3.2).
- **Ciphertext:** $k \cdot n \cdot \frac{8.008 + \log_2 t - 1}{8}$ for the C_u part, plus $n \cdot \frac{l + \log_2 t}{8}$ for the C_v part, where l is the number of bits to which the $\mathbb{Z}_q$ component of C_v is compressed.
- **Secret key:** For each of the k polynomials, the secret key stores:
 1. a 2-byte counter indicating how many non-zero coefficients are present in the t -polynomial;
 2. a sparse representation of the t -polynomial (each non-zero term stored as a pair (index, value), requiring 2 bytes per term);
 3. a dense representation of the q -polynomial, stored using the lossless coding of Algorithms 1 and 2 (hence occupying $n \times 8.008/8 \approx n$ bytes, rounded up to the nearest integer; for $n = 512$ this is 513 bytes in practice, but we use 512 bytes for simplicity by packing the 9-bit values into 8-bit storage with a small overhead – the exact figures are given in Table 2).

Consequently, the total secret-key size is $k \times (2 + 512) + TotalHWT \times 2$ bytes, where $TotalHWT$ is the total number of non-zero coefficients across all k t -polynomials.

The concrete values for n , k , t , l , the rounding set sizes, the fixed-weight counts, the error-correcting code strategy, the resulting DFR, and the estimated classical and quantum security levels are listed in Table 2. The hardness estimates were obtained using the Lattice Estimator [2] with the MATZOV [38] and Chailloux–Loyer [12] cost models for classical and quantum security, respectively.

5.2 Performance Analysis

We evaluate the performance of the reference implementation of Lore. Cycle counts were measured using the CPU’s timestamp counter; we report both the median and the average over 10,000 executions of each function. The reference implementation and the security estimation scripts are publicly available at <https://github.com/pq-CUC/Lore>.

Table 2. Parameters for the Lore PKE/KEM Scheme

Security Level λ	128	256	384	512
n	512	512	512	768
κ	128	256	384	512
k	1	2	3	3
t	2	2	4	4
$ S_R $	1	1	2	2
l	1	1	1	1
Secret Distribution	FW [1]	FW [2]	FW [3]	FW [4]
ECC Strategy [5]	$k_{vec} = 4$	(512,256,28)	(512,384,14)	(768,512,25)
DFR ($\log_2$)	-130.78	-259.28	-395.97	-528.55
BKZ block-size (Classical)	356	837	1258	1952
Classical Hardness (bits)	131.09	265.77	384.35	580.32
BKZ block-size (Quantum)	379	856	1282	1982
Quantum Hardness (bits)	97.41	220.53	329.53	509.47
Ciphertext size	641	1153	1921	2886
Public key size	545	1058	1763	2626
Secret key size	821	1942	3704	5373

Note [1]: For a secret vector with $k = 1$, the total coefficient counts across all three polynomials for values $\{-2, -1, 0, 1, 2\}$ are exactly $\{1, 50, 410, 50, 1\}$.

Note [2]: For a secret vector with $k = 2$, the total coefficient counts across all three polynomials for values $\{-2, -1, 0, 1, 2\}$ are exactly $\{12, 140, 720, 140, 12\}$.

Note [3]: For a secret vector with $k = 3$, the total coefficient counts across all four polynomials for values $\{-2, -1, 0, 1, 2\}$ are exactly $\{60, 270, 876, 270, 60\}$.

Note [4]: For a secret vector with $k = 3$, the total coefficient counts across all four polynomials for values $\{-2, -1, 0, 1, 2\}$ are exactly $\{60, 420, 1344, 420, 60\}$.

Note [5]: For achieving 128-bit security, we employ repetition code with $k_{vec} = 4$. For achieving 256-bit, 384-bit, and 512-bit security, we employ BCH codes. The parameters (n, κ, τ) of a BCH code denote the codeword length, message length, and error-correcting capability, respectively.

At the theoretical level, the main overhead of loreKEM during the key generation phase is the generation of the random matrix $\mathbf{A}$, one matrix-vector NTT operation modulo q , and one matrix-vector multiplication modulo t ; during the encapsulation phase, the main overhead is the generation of the random matrix $\mathbf{A}$, two matrix-vector NTT operations modulo q , two matrix-vector multiplications modulo t , and one encoding operation; during the decapsulation phase, the main overhead is the generation of the random matrix $\mathbf{A}$, three matrix-vector NTT operations modulo q , three matrix-vector multiplications modulo t , and one decoding operation.

Tables 3 and 4 compare the performance and bandwidth of Lore and Kyber [6] on a generic x64 platform. For a fair comparison, we used the same benchmarking environment described below.

As shown in Table 3, Lore achieves significantly smaller combined public-key and ciphertext sizes than Kyber at all security levels (especially at high security levels, supporting our observation that a fixed modulus inherently limits bandwidth optimization), while simultaneously attaining a DFR tightly aligned with the target security strength (e.g., 2^{-130} for 128-bit security and 2^{-266} for 256-bit security). The bandwidth reduction ranges from 25% to 30% compared to Kyber, confirming the effectiveness of CRT Compression.

In terms of computational cost, as shown in Table 4, Lore is computationally slower than Kyber in raw cycle counts due to the separate arithmetic in $\mathcal{R}_q$ and $\mathcal{R}_t$. However, when the estimated transmission cost of 2,000 cycles per byte (as suggested by the NIST status report [1]) is factored into the total cost, Lore’s substantial bandwidth advantage more than compensates for its computational overhead. Comparing equivalent security levels, Lore demonstrates a clear advantage: at the 128-bit security level, Lore-128 reduces the total cost by approximately 20% compared to Kyber-512; at the 256-bit security level, Lore-256 outperforms Kyber-1024 by roughly 24%. This confirms that for typical network scenarios, Lore’s compact bandwidth effectively translates into highly competitive overall performance, outperforming comparable schemes despite its modest computational cost. Besides, we summary the performance of the SM3 version with the experiment results shown in Table 5 by utilizing the function interface provided by NGCC.

The reference implementation of Lore KEM (using SHAKE/SM3 with no architecture-specific optimizations) was compiled and executed on an x86_64 machine running Ubuntu 20.04.6 LTS (Linux kernel 5.15.0-139-generic) with GCC 9.4.0. The CPU clock speed was 2.9 GHz. For each security level (Lore-L1 through Lore-L4), the build process consists of navigating to the corresponding subdirectory (e.g., Lore-L1) and running `make clean && make KAT_KEM_<level> && ./KAT_KEM_<level>`, where `<level>` is 1, 2, 3, or 4. The compiler flags used for all builds are `-O3 -DNDEBUG -D_GNU_SOURCE -std=gnu11 -D_POSIX_C_SOURCE=199309L`.

Deterministic known-answer tests (KAT) are generated using the provided `KAT_KEM.c`, `drng.c`, and auxiliary files; the output must match the test vectors in `Test_Vectors/KAT_KEM_Lore-Lx.txt`. Performance benchmarking of the IND-CPA primitives (`indcpa_keypair`, `indcpa_enc`, `indcpa_dec`) was performed on the same system, pinning the process to core 0 (`CORE=0`) with 1,000 warm-up iterations and 10,000 measurement iterations, using the `rdtsc.x86` cycle counter. Although the machine is equipped with an NVIDIA GeForce RTX 3090 GPU, all cycle measurements are CPU-only; the GPU was not used for any PQC operation.

To further evaluate scalability across platforms, we present test results on an ARM platform in Table 6. The test platform is an Ubuntu 22.04.4 LTS system running on aarch64 architecture (Linux kernel 5.15.0-94-generic) with a HiSilicon 8-core CPU (2.9GHz, 1 socket, 8 cores, no SMT), equipped with 15 GiB of RAM and caches of 512 KiB L1d, 512 KiB L1i, 4 MiB L2, and 32 MiB L3. The compiler used is GCC 11.4.0. Benchmarking is performed with the CPU core pinned to core 0, a warm-up phase of 1,000 iterations, and 10,000 measurement iterations; cycle counts are obtained via `perf_event_open_aarch64` (with `perf_event_paranoid` set to 4). The results show that both Lore and Kyber achieve a speedup of about 30% on the ARM platform, which further enlarges the total-cost advantage of Lore over Kyber to 21%–25%, indicating that Lore is highly scalable.

Parameter set	DFR	Ciphertext	Public key	Secret key	CT+PK
Lore-128	-128.81	641	545	821	1186
Lore-256	-266.53	1153	1058	1942	2211
Lore-384	-387.20	1921	1763	3704	3684
Lore-512	-521.75	2886	2626	5373	5514
Kyber-512	-139	800	768	768	1568
Kyber-768	-164	1184	1088	1152	2272
Kyber-1024	-174	1568	1568	1536	3136

Table 3. Ciphertext, key sizes (bytes) and DFR of Lore KEM(SHAKE version), and Kyber KEM.

References

1. Alagic, G., Apon, D., Cooper, D., et al.: NIST IR 8413: Status report on the third round of the nist post-quantum cryptography standardization process (2022), <https://csrc.nist.gov/pubs/ir/8413/upd1/final>
2. Albrecht, M.R., Player, R., Scott, S.: On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology*. Volume 9, Issue 3, Pages 169–203 (2015), <https://github.com/malb/lattice-estimator>

Parameter set		KeyGen	Encaps	Decaps	Total Cost*
Lore-128	med	87087	164378	219231	2842696
	avg	91697	164957	220044	2848698
Lore-256	med	283950	397372	542431	5645753
	avg	285104	401327	547760	5656193
Kyber-512	med	99866	125054	149980	3510900
	avg	103617	125938	151315	3516871
Kyber-1024	med	253332	281202	321592	7128126
	avg	256735	282734	323259	7134729

Table 4. Median and average cycle counts of 10000 executions for Lore KEM (SHAKE version) and Kyber KEM on x86_64 platform

*Total cost is estimated by $2000 \text{ cycles/byte} \times (|\text{PK}| + |\text{CT}|) + \text{KeyGen} + \text{Encaps} + \text{Decaps}$ [1].

Parameter set		KeyGen	Encaps	Decaps	Total Cost*
Lore-128	med	166632	255490	311560	3105682
	avg	170526	259055	315980	3117562
Lore-256	med	545508	690437	826231	6484176
	avg	551661	697312	833533	6504507
Lore-384	med	1223414	1459146	1635923	11686483
	avg	1227072	1480140	1653424	11728637
Lore-512	med	2785019	3258352	3584699	20652070
	avg	2788975	3279248	3608512	20700736

Table 5. Median and average cycle counts of 10000 executions for Lore KEM (SM3 version) on x86_64 platform

*Total cost is estimated by $2000 \text{ cycles/byte} \times (|\text{PK}| + |\text{CT}|) + \text{KeyGen} + \text{Encaps} + \text{Decaps}$ [1].

Parameter set		KeyGen	Encaps	Decaps	Total Cost*
Lore-128	med	59065	116548	157013	2704626
	avg	59126	116644	157238	2705008
Lore-256	med	184796	266190	336024	5209010
	avg	184916	266456	336671	5210043
Kyber-512	med	76815	94382	114610	3421807
	avg	76984	95510	114830	3423324
Kyber-1024	med	203765	233101	268626	6977492
	avg	204013	233277	268883	6978173

Table 6. Median and average cycle counts of 10000 executions for Lore KEM (SHAKE version) and Kyber KEM on ARM platform

*Total cost is estimated by $2000 \text{ cycles/byte} \times (|\text{PK}| + |\text{CT}|) + \text{KeyGen} + \text{Encaps} + \text{Decaps}$ [1].

3. Alkim, E., Ducas, L., Pöppelmann, T., Schwabe, P.: Post-quantum key exchange - A new hope. In: Holz, T., Savage, S. (eds.) 25th USENIX Security Symposium, USENIX Security 16, Austin, TX, USA, August 10-12, 2016. pp. 327–343. USENIX Association (2016)
4. Alwen, J., Krenn, S., Pietrzak, K., Wichs, D.: Learning with rounding, revisited - new reduction, properties and applications. In: Canetti, R., Garay, J.A. (eds.) Advances in Cryptology - CRYPTO 2013 - 33rd Annual Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2013. Proceedings, Part I. Lecture Notes in Computer Science, vol. 8042, pp. 57–74. Springer (2013). https://doi.org/10.1007/978-3-642-40041-4_4
5. Applebaum, B., Cash, D., Peikert, C., Sahai, A.: Fast cryptographic primitives and circular-secure encryption based on hard learning problems. In: Halevi, S. (ed.) Advances in Cryptology - CRYPTO 2009, 29th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2009. Proceedings. Lecture Notes in Computer Science, vol. 5677, pp. 595–618. Springer (2009). https://doi.org/10.1007/978-3-642-03356-8_35
6. Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehlé, D.: Crystals-kyber – algorithm specifications and supporting documentation (version 3.02). Tech. rep. (Sep 2021), <https://pq-crystals.org/kyber/resources.shtml>, final round specification document submitted to NIST PQC standardization
7. Banerjee, A., Peikert, C., Rosen, A.: Pseudorandom functions and lattices. In: Pointcheval, D., Johansson, T. (eds.) Advances in Cryptology - EUROCRYPT 2012 - 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cambridge, UK, April 15-19, 2012. Proceedings. Lecture Notes in Computer Science, vol. 7237, pp. 719–737. Springer (2012). https://doi.org/10.1007/978-3-642-29011-4_42
8. Becker, A., Ducas, L., Gama, N., Laarhoven, T.: New directions in nearest neighbor searching with applications to lattice sieving. In: Krauthgamer, R. (ed.) Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2016, Arlington, VA, USA, January 10-12, 2016. pp. 10–24. SIAM (2016). <https://doi.org/10.1137/1.9781611974331.CH2>
9. Bi, L., Lu, X., Luo, J., Wang, K.: Hybrid dual and meet-lwe attack. In: Nguyen, K., Yang, G., Guo, F., Susilo, W. (eds.) Information Security and Privacy - 27th Australasian Conference, ACISP 2022, Wollongong, NSW, Australia, November 28-30, 2022, Proceedings. Lecture Notes in Computer Science, vol. 13494, pp. 168–188. Springer (2022). https://doi.org/10.1007/978-3-031-22301-3_9
10. Bindel, N., Hamburg, M., Hövelmanns, K., Hülsing, A., Persichetti, E.: Tighter proofs of CCA security in the quantum random oracle model. In: Hofheinz, D., Rosen, A. (eds.) Theory of Cryptography - 17th International Conference, TCC 2019, Nuremberg, Germany, December 1-5, 2019, Proceedings, Part II. Lecture Notes in Computer Science, vol. 11892, pp. 61–90. Springer (2019). https://doi.org/10.1007/978-3-030-36033-7_3
11. Bindel, N., Schanck, J.M.: Decryption failure is more likely after success. In: Ding, J., Tillich, J. (eds.) Post-Quantum Cryptography - 11th International Conference, PQCrypto 2020, Paris, France, April 15-17, 2020, Proceedings. Lec-

- ture Notes in Computer Science, vol. 12100, pp. 206–225. Springer (2020). https://doi.org/10.1007/978-3-030-44223-1_12
12. Chailloux, A., Loyer, J.: Lattice sieving via quantum random walks. In: Tibouchi, M., Wang, H. (eds.) *Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security*, Singapore, December 6–10, 2021, Proceedings, Part IV. Lecture Notes in Computer Science, vol. 13093, pp. 63–91. Springer (2021). https://doi.org/10.1007/978-3-030-92068-5_3
 13. Chen, Y., Nguyen, P.Q.: BKZ 2.0: Better lattice security estimates. In: Lee, D.H., Wang, X. (eds.) *Advances in Cryptology - ASIACRYPT 2011 - 17th International Conference on the Theory and Application of Cryptology and Information Security*, Seoul, South Korea, December 4–8, 2011. Proceedings. Lecture Notes in Computer Science, vol. 7073, pp. 1–20. Springer (2011). https://doi.org/10.1007/978-3-642-25385-0_1
 14. Cheon, J.H., Choe, H., Seo, J., Seong, H.: SMAUG (-t), revisited: Timing-secure, more compact, less failure. *IEEE Access* **12**, 188386–188397 (2024). <https://doi.org/10.1109/ACCESS.2024.3511346>
 15. Cini, V., Ramacher, S., Slamanig, D., Striecks, C.: Cca-secure (puncturable) kems from encryption with non-negligible decryption errors. In: *International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT)*. pp. 159–190. Springer (2020). https://doi.org/10.1007/978-3-030-64837-4_6
 16. D’Anvers, J., Guo, Q., Johansson, T., Nilsson, A., Vercauteren, F., Verbauwheide, I.: Decryption failure attacks on IND-CCA secure lattice-based schemes. In: Lin, D., Sako, K. (eds.) *Public-Key Cryptography - PKC 2019 - 22nd IACR International Conference on Practice and Theory of Public-Key Cryptography*, Beijing, China, April 14–17, 2019, Proceedings, Part II. Lecture Notes in Computer Science, vol. 11443, pp. 565–598. Springer (2019). https://doi.org/10.1007/978-3-030-17259-6_19
 17. D’Anvers, J., Karmakar, A., Roy, S.S., Vercauteren, F.: Saber: Module-lwr based key exchange, cpa-secure encryption and cca-secure KEM. In: Joux, A., Nitaj, A., Rachidi, T. (eds.) *Progress in Cryptology - AFRICACRYPT 2018 - 10th International Conference on Cryptology in Africa*, Marrakesh, Morocco, May 7–9, 2018, Proceedings. Lecture Notes in Computer Science, vol. 10831, pp. 282–305. Springer (2018). https://doi.org/10.1007/978-3-319-89339-6_16
 18. D’Anvers, J.P., Karmakar, A., Roy, S.S., Vercauteren, F.: Saber: Mod-LWR based kem. Tech. rep. (2017)
 19. Espitau, T., Joux, A., Kharchenko, N.: On a dual/hybrid approach to small secret LWE - A dual/enumeration technique for learning with errors and application to security estimates of FHE schemes. In: Bhargavan, K., Oswald, E., Prabhakaran, M. (eds.) *Progress in Cryptology - INDOCRYPT 2020 - 21st International Conference on Cryptology in India*, Bangalore, India, December 13–16, 2020, Proceedings. Lecture Notes in Computer Science, vol. 12578, pp. 440–462. Springer (2020). https://doi.org/10.1007/978-3-030-65277-7_20

20. Fouque, P.A., Kirchner, P., Pornin, T., Yu, Y.: Bat: Small and fast kem over ntru lattices. *IACR Transactions on Cryptographic Hardware and Embedded Systems* pp. 240–265 (2022)
21. Fujisaki, E., Okamoto, T.: Secure integration of asymmetric and symmetric encryption schemes. *Journal of cryptology* **26**(1), 80–101 (2013)
22. Guo, Q., Johansson, T., Nilsson, A.: A generic attack on lattice-based schemes using decryption errors with application to ss-ntru-pke. *IACR Cryptol. ePrint Arch.* p. 43 (2019)
23. Guo, Q., Johansson, T., Yang, J.: A novel CCA attack using decryption errors against LAC. In: Galbraith, S.D., Moriai, S. (eds.) *Advances in Cryptology - ASIACRYPT 2019 - 25th International Conference on the Theory and Application of Cryptology and Information Security*, Kobe, Japan, December 8–12, 2019, Proceedings, Part I. *Lecture Notes in Computer Science*, vol. 11921, pp. 82–111. Springer (2019). https://doi.org/10.1007/978-3-030-34578-5_4
24. Hanrot, G., Pujol, X., Stehlé, D.: Analyzing blockwise lattice algorithms using dynamical systems. In: Rogaway, P. (ed.) *Advances in Cryptology - CRYPTO 2011 - 31st Annual Cryptology Conference*, Santa Barbara, CA, USA, August 14–18, 2011. Proceedings. *Lecture Notes in Computer Science*, vol. 6841, pp. 447–464. Springer (2011). https://doi.org/10.1007/978-3-642-22792-9_25
25. Hofheinz, D., Hövelmanns, K., Kiltz, E.: A modular analysis of the fujisaki-okamoto transformation. In: Kalai, Y., Reyzin, L. (eds.) *Theory of Cryptography - 15th International Conference, TCC 2017*, Baltimore, MD, USA, November 12–15, 2017, Proceedings, Part I. *Lecture Notes in Computer Science*, vol. 10677, pp. 341–371. Springer (2017). https://doi.org/10.1007/978-3-319-70500-2_12
26. Liu, Y., Zhang, Y., Lu, X., Cheng, Y., Yin, Y.: Dawn: Smaller and faster ntru encryption via double encoding. In: *International Conference on the Theory and Application of Cryptology and Information Security*. pp. 396–427. Springer (2025)
27. Lu, X., Liu, Y., Zhang, Z., Jia, D., Xue, H., He, J., Li, B.: LAC: practical Ring-LWE based public-key encryption with byte-level modulus. *IACR Cryptology ePrint Archive* **2018**, 1009 (2018)
28. Lu, X., Liu, Y., Zhang, Z., Jia, D., Xue, H., He, J., Li, B., Wang, K.: Lac: Practical ring-lwe based public-key encryption with byte-level modulus. *Cryptology ePrint Archive* (2018)
29. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. In: *Advances in Cryptology - EUROCRYPT 2010, 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Monaco / French Riviera, May 30 - June 3, 2010. Proceedings. pp. 1–23 (2010)
30. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. *J. ACM* **60**(6), 43:1–43:35 (2013). <https://doi.org/10.1145/2535925>
31. Lyubashevsky, V., Seiler, G.: Ntru: truly fast ntru using ntt. *IACR Transactions on Cryptographic Hardware and Embedded Systems* pp. 180–201 (2019)
32. NICCS: Submission requirements for public-key cryptographic algorithms (2025), <https://www.niccs.org.cn/niccs/Notice/1975896137741635584/IDop1mav.pdf>
33. NIST: Submission requirements and evaluation criteria for the post-quantum cryptography standardization process (2017), <https://csrc.nist.gov/Projects/post->

quantum-cryptography/post-quantum-cryptography-standardization/Call-for-Proposals

34. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. In: Gabow, H.N., Fagin, R. (eds.) *Proceedings of the 37th Annual ACM Symposium on Theory of Computing*, Baltimore, MD, USA, May 22-24, 2005. pp. 84–93. ACM (2005)
35. Schnorr, C., Euchner, M.: Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Math. Program.* **66**, 181–199 (1994). <https://doi.org/10.1007/BF01581144>
36. Son, Y., Cheon, J.H.: Revisiting the hybrid attack on sparse secret LWE and application to HE parameters. In: Brenner, M., Lepoint, T., Rohloff, K. (eds.) *Proceedings of the 7th ACM Workshop on Encrypted Computing & Applied Homomorphic Cryptography, WAHC@CCS 2019*, London, UK, November 11-15, 2019. pp. 11–20. ACM (2019). <https://doi.org/10.1145/3338469.3358941>
37. of Standards, N.I., (NIST), T.: Module-lattice-based key-encapsulation mechanism standard. Tech. Rep. FIPS 203, U.S. Department of Commerce, Washington, D.C. (2024). <https://doi.org/10.6028/NIST.FIPS.203>, <https://csrc.nist.gov/pubs/fips/203/final>
38. The MATZOV Researchers, The Center of Encryption and Information Security: Report on the security of lwe: Improved dual lattice attack. Tech. rep., Israel Defense Forces (IDF) (April 2022), available at <https://zenodo.org/records/6493704>
39. Wang, T., Wang, A., Wang, X.: Exploring decryption failures of BIKE: new class of weak keys and key recovery attacks. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023*, Santa Barbara, CA, USA, August 20-24, 2023, *Proceedings, Part III*. Lecture Notes in Computer Science, vol. 14083, pp. 70–100. Springer (2023). https://doi.org/10.1007/978-3-031-38548-3_3
40. Zhang, J., Feng, D., Yan, D.: Nev: faster and smaller ntru encryption using vector decoding. In: *International Conference on the Theory and Application of Cryptology and Information Security*. pp. 157–189. Springer (2023)
41. Zhang, J., Yu, Y., Fan, S., Zhang, Z., Yang, K.: Tweaking the asymmetry of asymmetric-key cryptography on lattices: Kems and signatures of smaller sizes. In: *IACR International Conference on Public-Key Cryptography*. pp. 37–65. Springer (2020)